Dr. Hans-Peter Hoidn
Distinguished IT Architect (Open Group certified)



Enterprise IT Architectures

Technology Architecture, Information Systems Architecture and SOA (Service Oriented Architecture)

© 2014 Hans-Peter Hoidn & Kai Schwidder

Technology Architecture

© 2014 Hans-Peter Hoidn & Kai Schwidder

Technology Architecture – Purpose

- provides the information for the physical implementation
  - Mapping from logical to physical
  - Dealing with the distribution of components to nodes
- Is heavily influenced by Technology Development
  - Defines a Technology "Platform"
  - Provides the basis for the implementation of the solutions
- Patterns and Reference Architectures play a major role:
  - Reference Architectures are seen as Patterns
  - E.g. SOA Reference Architecture
  - E.g. Cloud Reference Architecture

© 2014 Hans-Peter Hoidn & Kai Schwidder

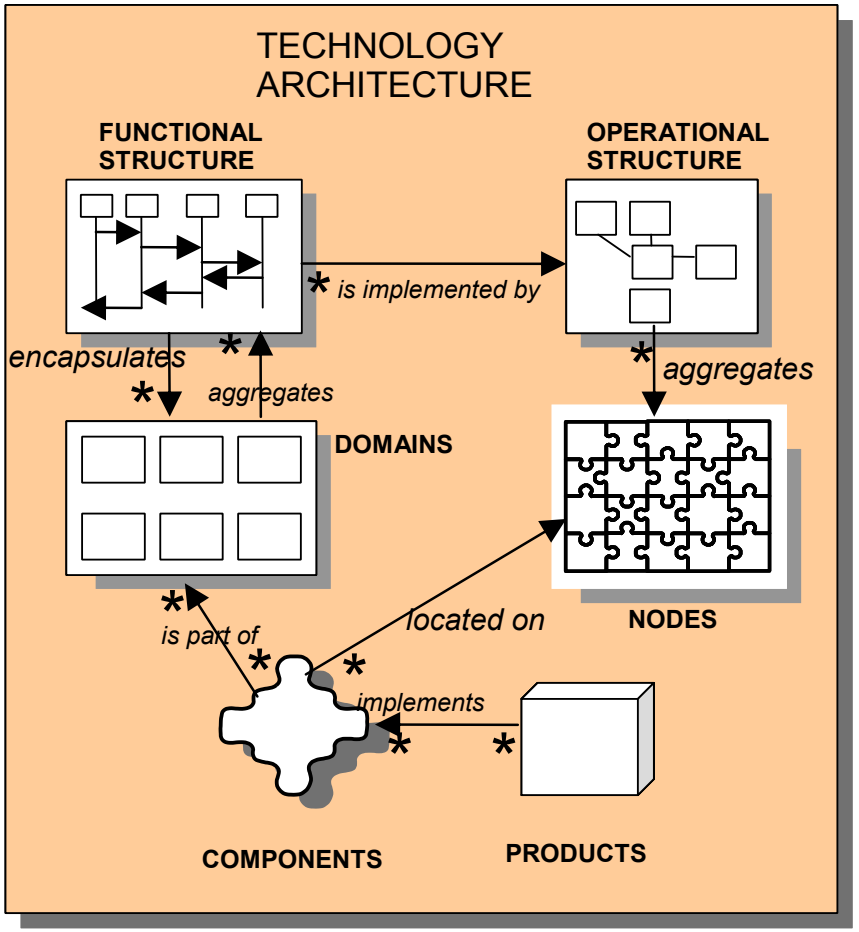
First Step for "*Technology Architecture*"
(some key words highlighted)

- **Review and validate the set of technology *principles*. These will normally form part of an overarching set of architecture principles**
- **Select relevant Technology Architecture resources (*reference models*, *patterns*, etc.)**
- **Select relevant Technology Architecture viewpoints that will enable the architect to demonstrate how the stakeholder concerns are being addressed in the Technology Architecture**
- **Identify appropriate tools and techniques to be used for capture, modeling, and analysis, in association with the selected viewpoints**

© 2014 Hans-Peter Hoidn & Kai Schwidder

Technology Architecture – Overview

- **Defines the essential guidance (rules) for selecting, building and maintaining a technical infrastructure composed of a set of hardware and software environments and platforms with defined interfaces, standards & services**
- **Defines the nodes, components, connections, collaborations and locations required**
- **Is based largely on proven architectural patterns, reference models or templates for the domains**
- **Provides design guidance, where appropriate, for developers**
- **Provides a framework for setting technology and product standards**



Source: IBM Architectural Thinking

© 2014 Hans-Peter Hoidn & Kai Schwidder

Technology Architecture – Guidance
described in accordance with four key aspects

Components

Technology "Building blocks" – often "standards"

Nodes

Logical Servers

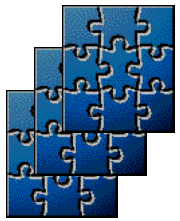
Logical Clients

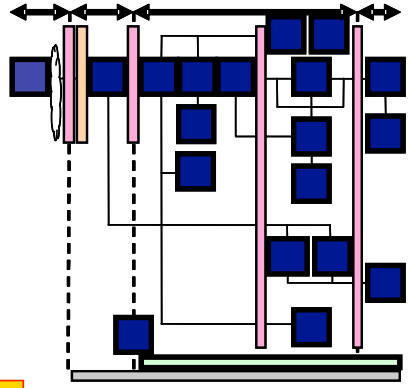
Operational Model

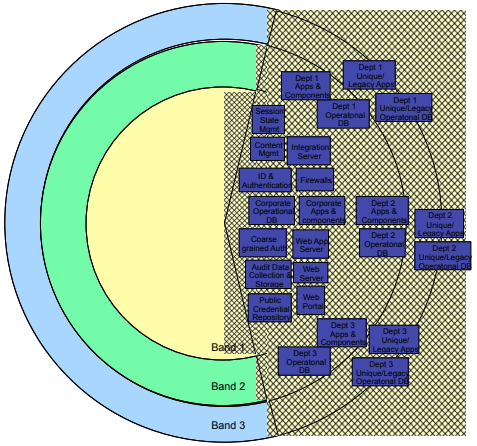
Assemblies of nodes required for common solutions

Placement:

Implementation guidance







Standards Criteria

E.g. RDBMS

ANSI SQL 92

E.g. Operational Database

E.g. Data Warehouse

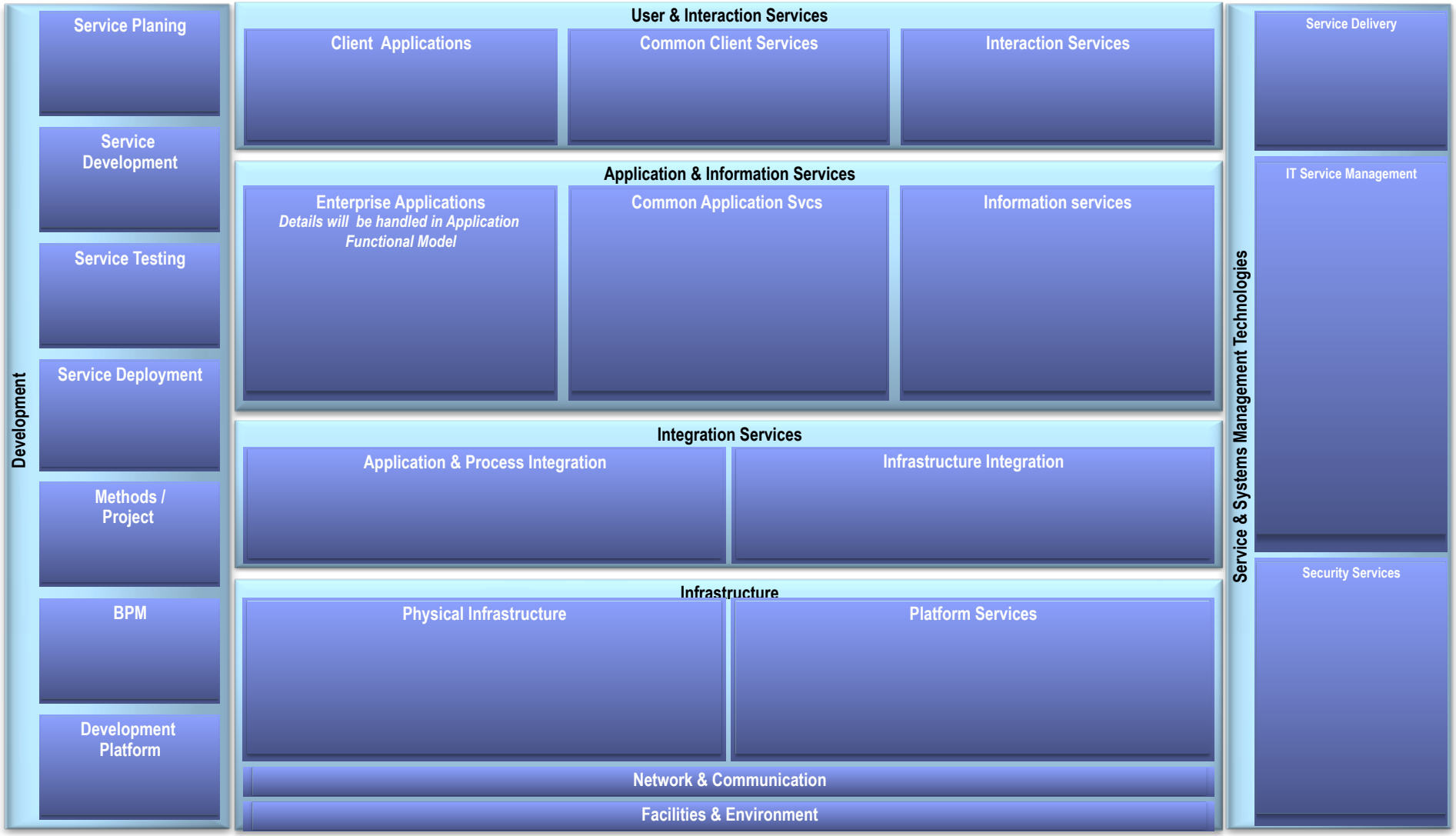
E.g. B2P Pattern

E.g. B2P Placement

Source: IBM Architectural Thinking

© 2014 Hans-Peter Hoidn & Kai Schwidder

Sample Technology Framework: Level 1



© 2014 Hans-Peter Hoidn & Kai Schwidder

Sample Technology Framework: Level 2



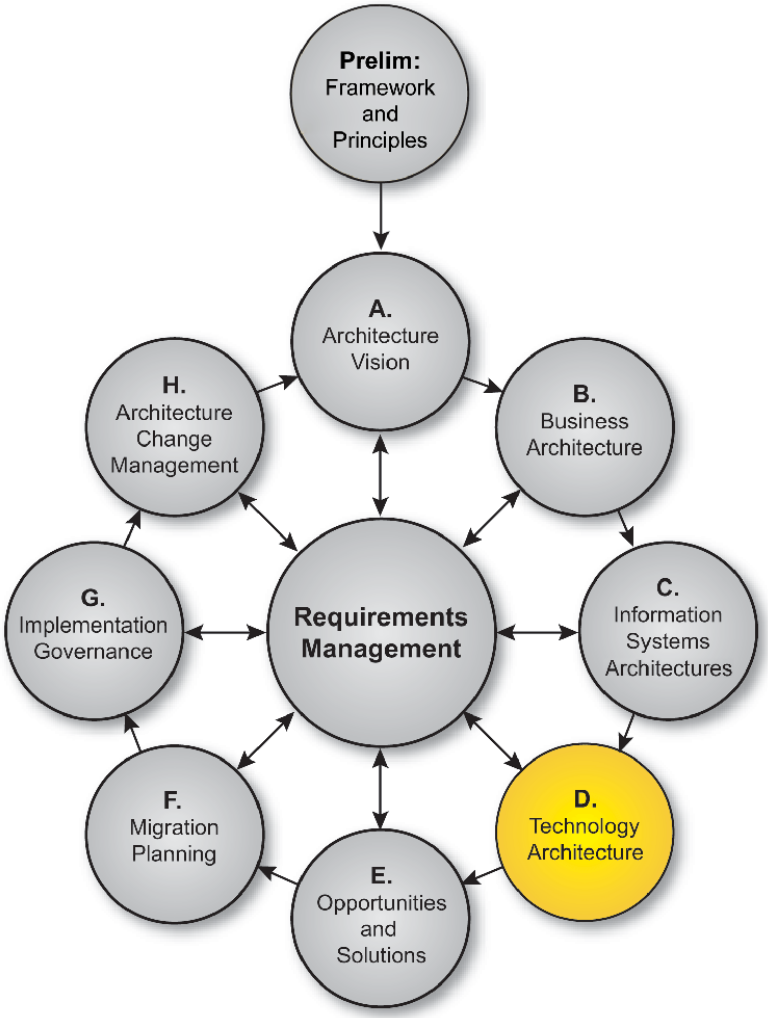
© 2014 Hans-Peter Hoidn & Kai Schwidder

Sample Technology Framework: Level 3



© 2014 Hans-Peter Hoidn & Kai Schwidder

Technology Architecture – Content according to TOGAF



- **The fundamental organization of an IT system, embodied in**
  - **its hardware, software and communications technology**
  - **their relationships to each other and the environment,**
  - **and the principles governing its design and evolution**

© 2014 Hans-Peter Hoidn & Kai Schwidder

Technology Architecture – Objectives (TOGAF 12.1)

- Develop the Target Technology Architecture that enables the logical and physical application and data components and the Architecture Vision, addressing the Request for Architecture Work and stakeholder concerns

- Identify candidate Architecture Roadmap components based upon gaps between the Baseline and Target Technology Architectures

© 2014 Hans-Peter Hoidn & Kai Schwidder

Technology Architecture – Steps (TOGAF 12.4)

- Select reference models, viewpoints, and tools
- Develop Baseline Technology Architecture Description
- Develop Target Technology Architecture Description
- Perform gap analysis
- Define candidate roadmap components
- Resolve impacts across the Architecture Landscape
- Conduct for mal stakeholder review
- Finalize the Technology Architecture
- Create Architecture Definition Document

© 2014 Hans-Peter Hoidn & Kai Schwidder

Technology Architecture – Topics (TOGAF Pocket Guide)

- Baseline Technology Architecture, if appropriate Target Technology Architecture, including:
  - Technology components and their relationships to information systems
  - Technology platforms and their decomposition, showing the combinations of
- Technology required to realize a particular technology "stack"
  - Environments and locations – a grouping of the required technology into computing environments (e.g., development, production)
  - Expected processing load and distribution of load across technology components — Physical(network)communications
  - Hardware and network specifications
  - Views corresponding to the selected viewpoints addressing key stakeholder concerns

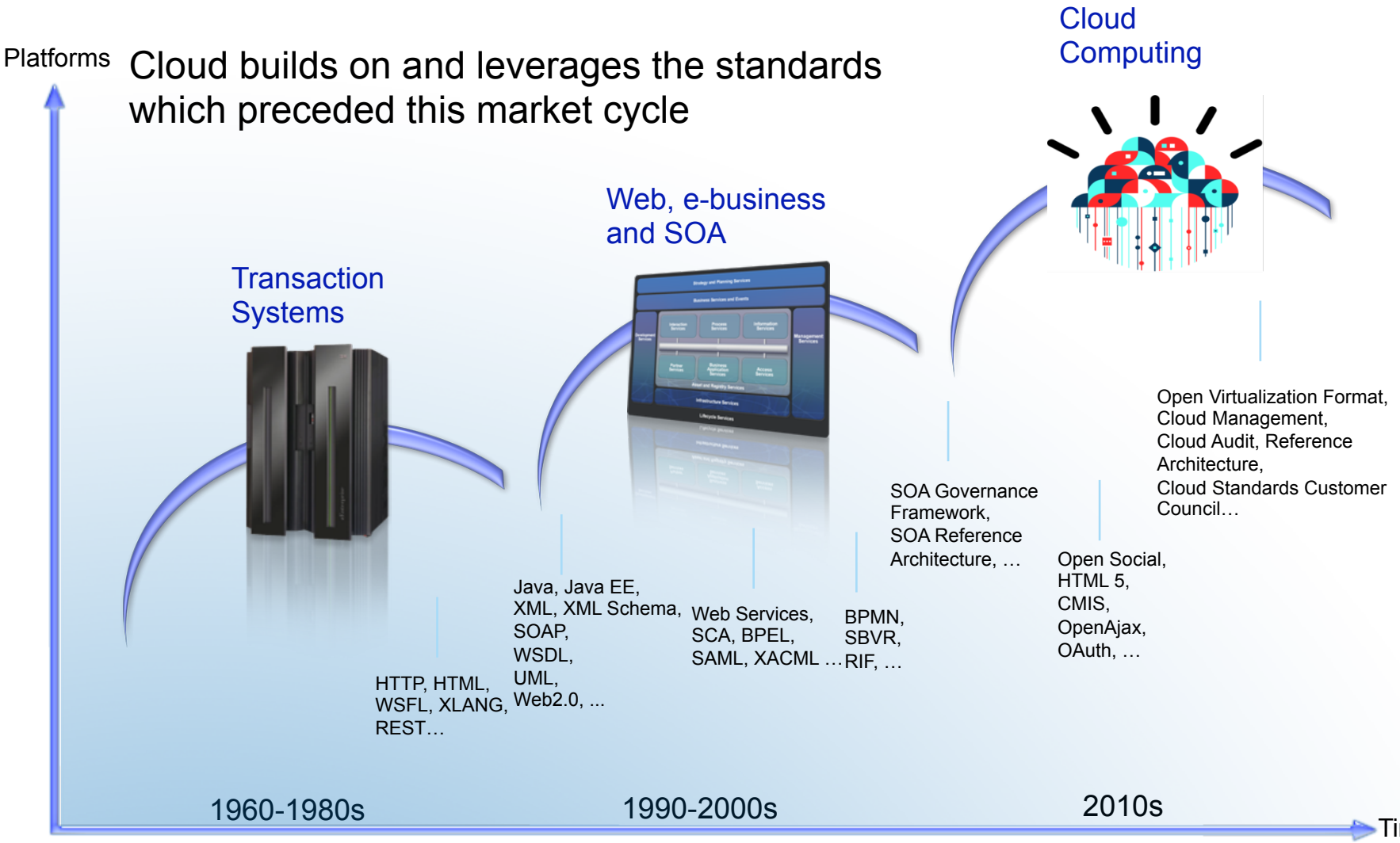
© 2014 Hans-Peter Hoidn & Kai Schwidder

Reference Architectures – Overview

- TOGAF
  - Is providing two reference architectures, one of those is the TRM (TOGAF Technical Reference Model)
  - TRM specifies various technical services, the details showing service categories, see TOGAF Chapter 43, pp. 491 – 521
  - If not a huge project, this taxonomy is too much !
- SOA Reference Architecture (as provided by IBM)
  - Provides appropriate basis for an Integration Platform, firstly logically but can be augmented by products
  - Much better suitable for “normal” usage and consulting
- Cloud Reference Architecture

© 2014 Hans-Peter Hoidn & Kai Schwidder

In Addition: Constantly shifts in Technology impact *Technology Reference Architectures*



© 2014 Hans-Peter Hoidn & Kai Schwidder

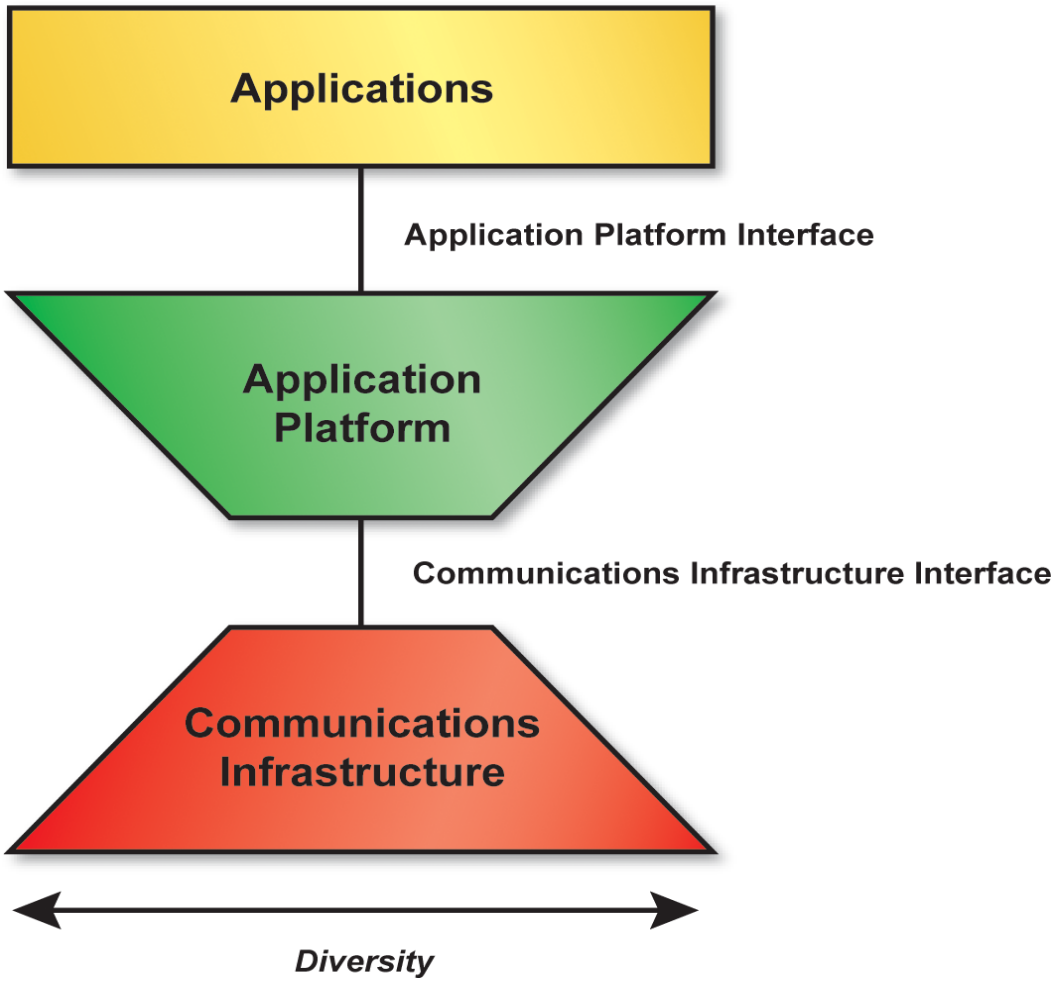
TOGAF® Reference Models

- The TOGAF Technical Reference Model (TRM)
  - A Foundation Architecture
  - A model and a taxonomy of generic platform services

- The Integrated Information Infrastructure Model (III-RM).
  - A model for business applications and infrastructure applications
  - Specifically aimed to support the vision of Boundaryless Information Flow™

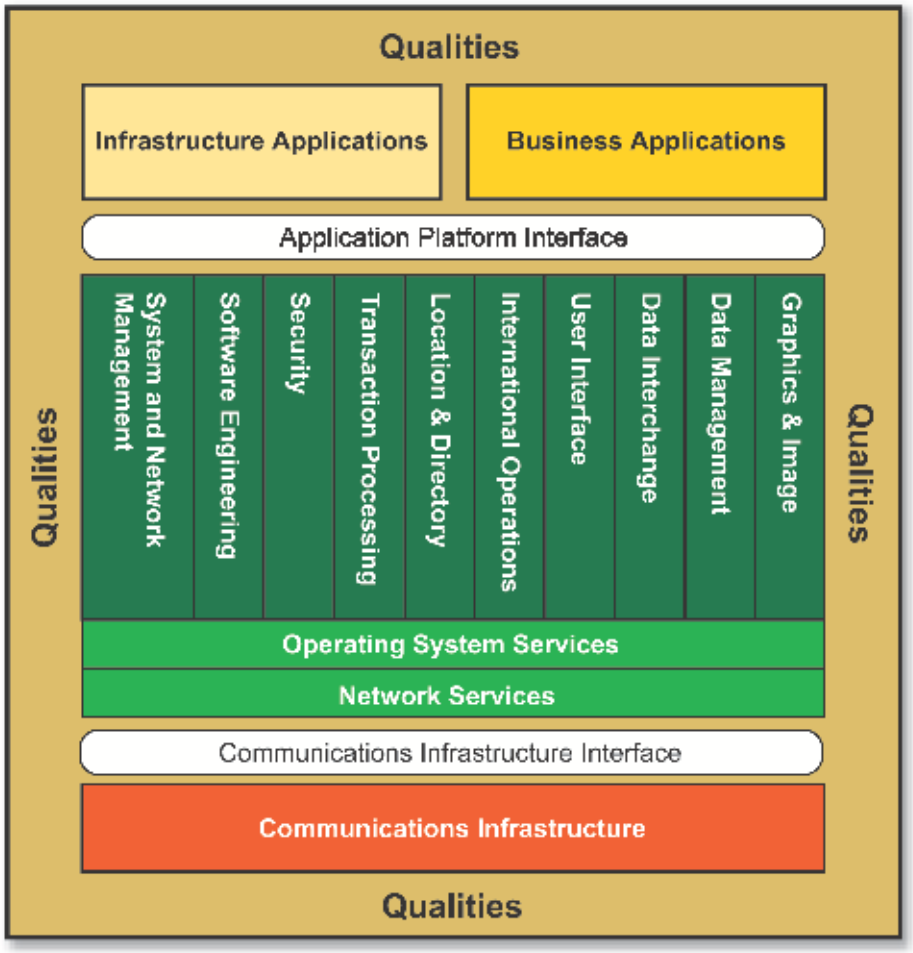
© 2014 Hans-Peter Hoidn & Kai Schwidder

TRM (TOGAF Technical Reference Model) High-Level View



© 2014 Hans-Peter Hoidn & Kai Schwidder

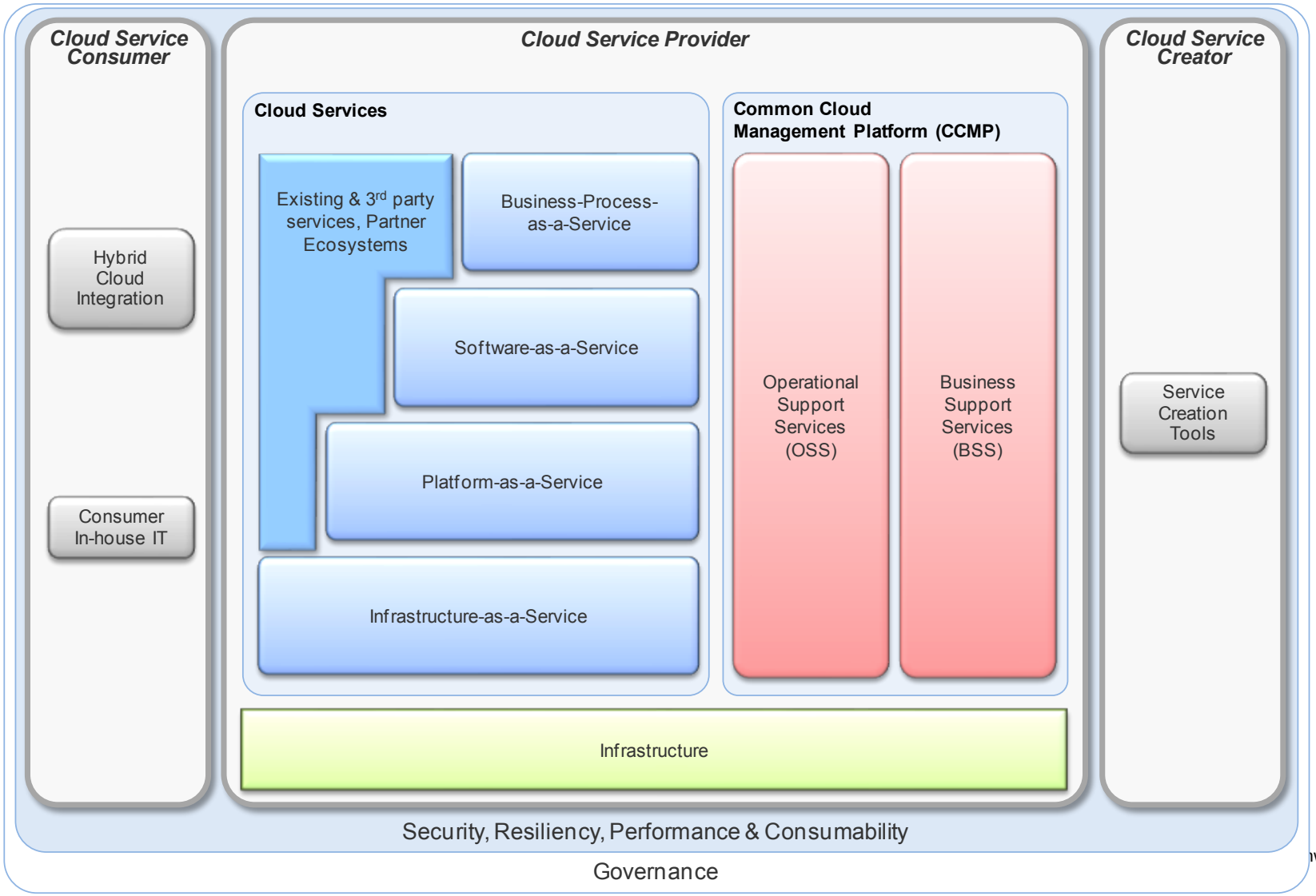
TRM (TOGAF Technical Reference Model) Detailed View



© 2014 Hans-Peter Hoidn & Kai Schwidder

Cloud Computing Reference Architecture (CCRA) Version 3.0

The CCRA provides a common, coherent architectural structure



Information Systems (IS) Architecture

© 2014 Hans-Peter Hoidn & Kai Schwidder

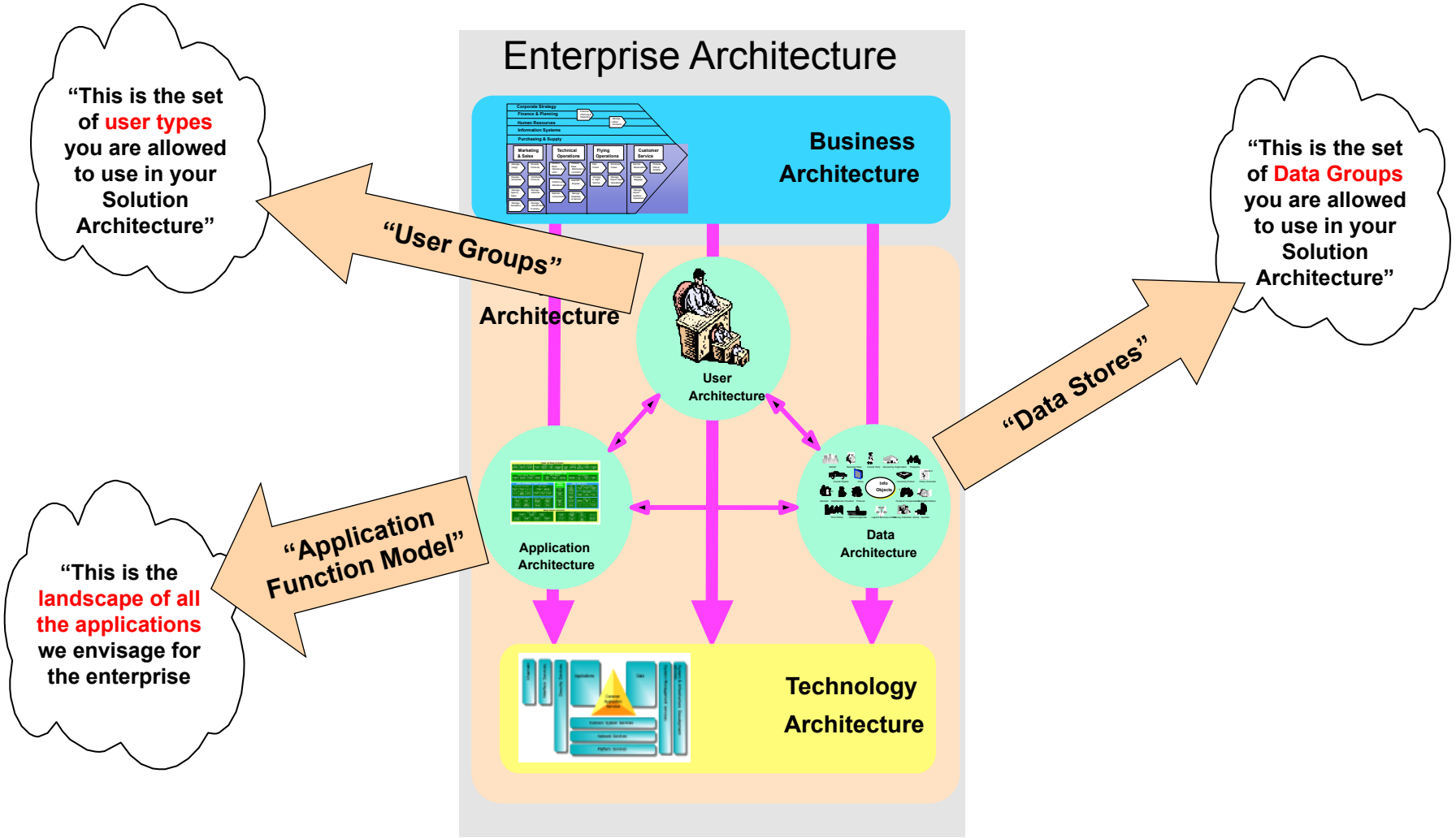
Information Systems (IS) Architecture – Overview

- The Information Systems Architecture provides the information about the buildings blocks to support the business
  - Objectives general addressing “application systems” and “data”
  - Capture “Baseline” and “Target”, different views

- Definition: An IS architecture describes aspects of business that are to be automated – known as the “business dependent IT architecture”. The IS Architecture manages
  - Functional Architecture
  - User Groups
  - Application Groups
  - Data Stores
  - Application Components
  - IS Services

© 2014 Hans-Peter Hoidn & Kai Schwidder

Information Systems (IS) Architecture – Illustration



© 2014 Hans-Peter Hoidn & Kai Schwidder

In more detail: There are two steps to defining IS Architecture: Step1: Select what functionality and information to automate…

- Understand currently automated business activities;
- Assess users groups, their roles and information requirements;
- Understand strategic decisions already made and being implemented;
  - Package selection decisions
  - Sourcing decisions
  - Programs, projects in flight
- Evaluate current portfolio against Business Architecture;
- Understand essential non functional requirements.
- Use Reference Architectures and Patterns as accelerators;
- Assess and prioritize gaps and challenges;

© 2014 Hans-Peter Hoidn & Kai Schwidder

In more detail: Step 2: Decide how to package and structure this automation to deliver measurable business value.

- **"Upstream EA" (Enterprise Level)**
  - Enterprise-wide picture of all automated functionality and information ("on-a-page")
  - Coarse grained components (e.g Account Management)
  - Minimal redundancy of functionality - potential for high levels of replaceability
  - Constraints for more granular (solution) level components
  - Loose coupling between components
  - Minimize technology dependencies
- **"Downstream EA" (Solution level)**
  - Set of building blocks (ABBs) at a level Solution Architects can use.
  - Finer grained components (e.g. create account, assess credit score, check background, etc)
  - No redundancy of functionality - potential for high levels of reusability
  - Loose coupling between components
  - High cohesion within components

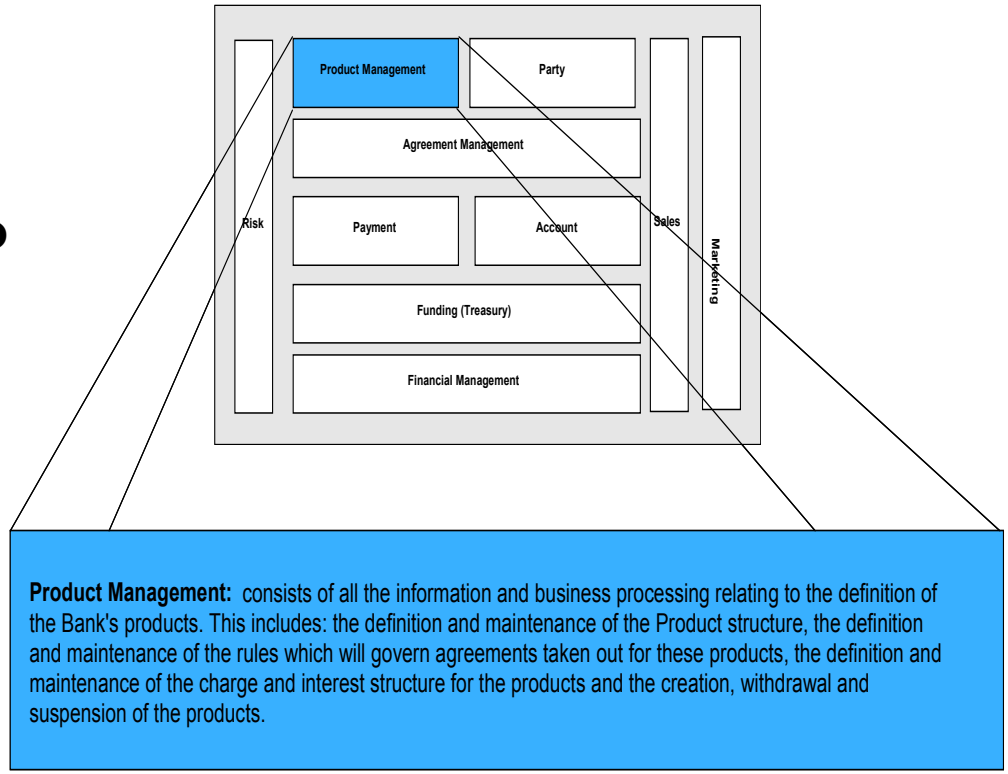


© 2014 Hans-Peter Hoidn & Kai Schwidder

Function analysis describes how business information, business activities and user roles will be grouped & partitioned into functionality (e.g. application groups or services)

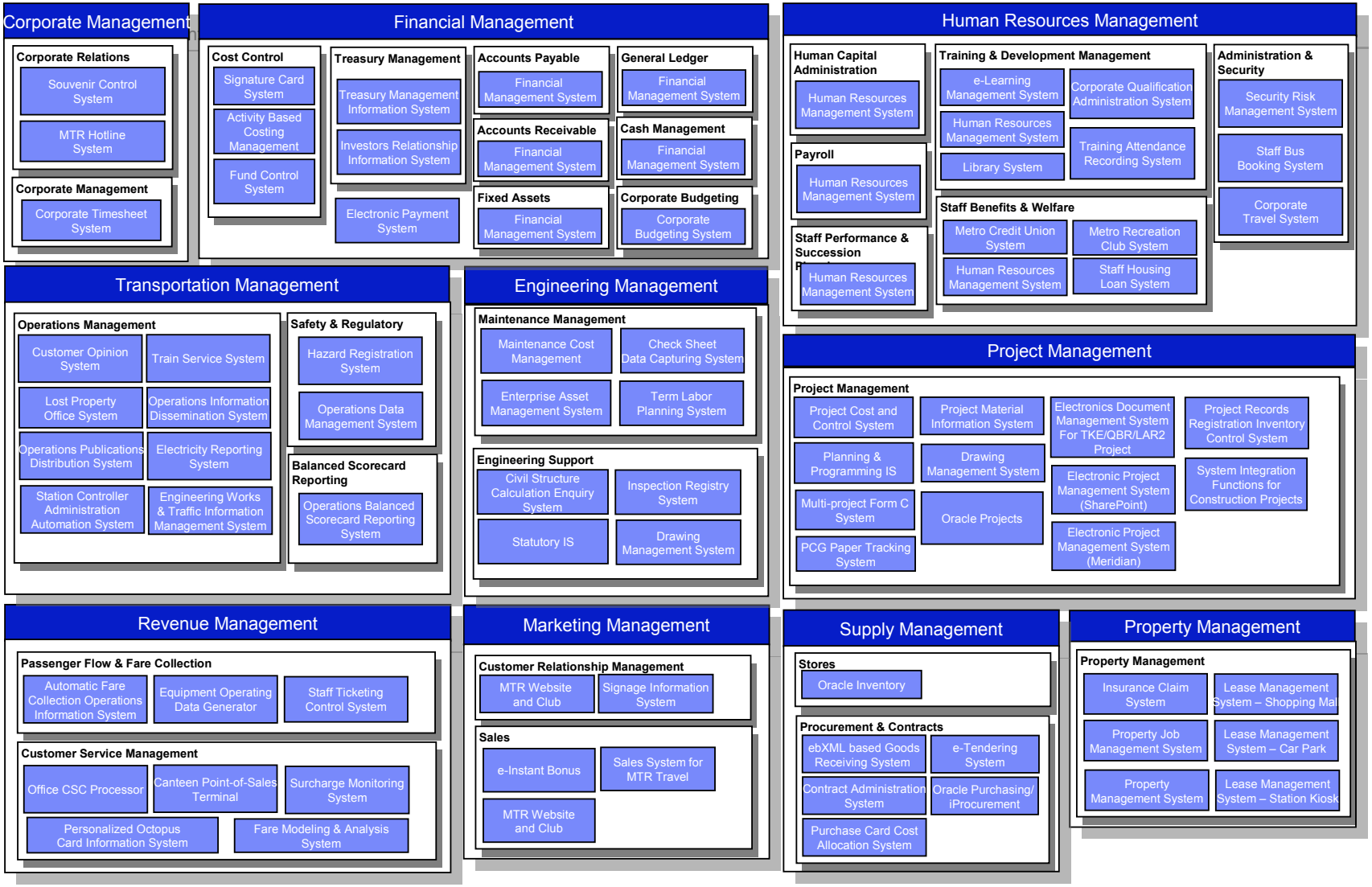
Functionality is optimal when:

- **Business functions are closely related;**
- **Scope and boundaries of each functions are clearly defined and do not overlap;**
- **Interface and interactions between functions are well-defined;**
- **Functions are able to access and share common data, rather than 'owning' data;**
- **Functions represent "reasonable" pieces of work that can be tackled by projects.**



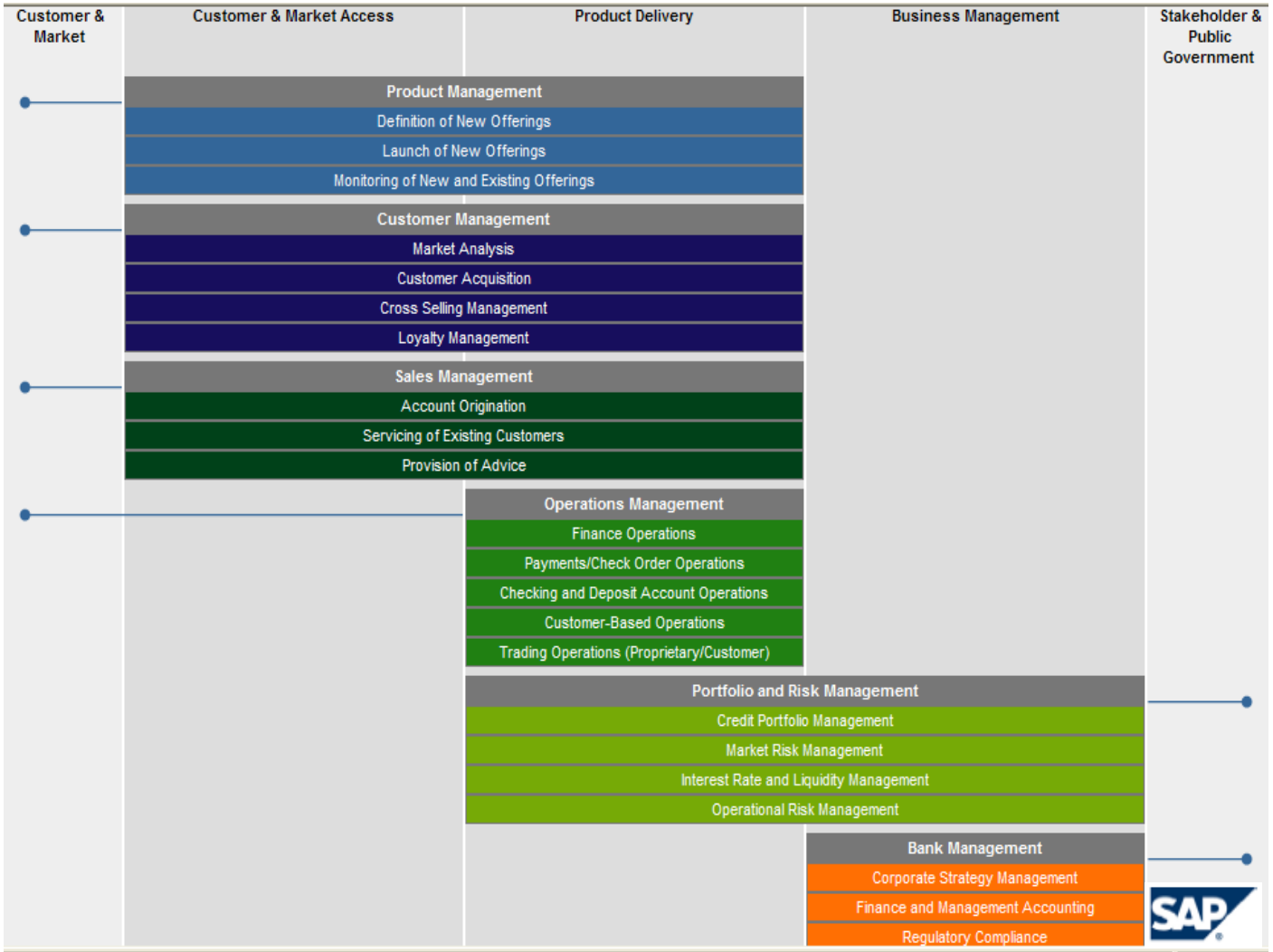
© 2014 Hans-Peter Hoidn & Kai Schwidder

Example of a IS Function Map



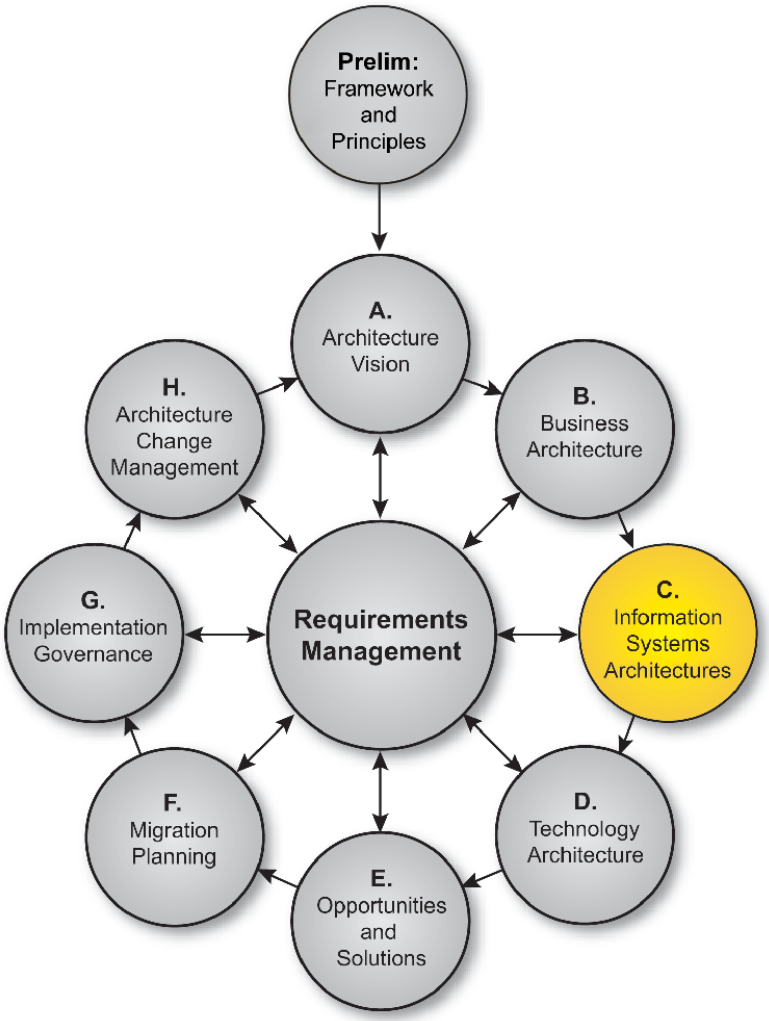
© 2014 Hans-Peter Hoidn & Kai Schwidder

Business application vendors have pre-built Functional/Process maps that can be used to accelerate the definition process.



© 2014 Hans-Peter Hoidn & Kai Schwidder

IS Architecture Content according to TOGAF



- The fundamental organization of an IT system, embodied in
  - relationships to each other and the environment, and the principles governing its design and evolution
- Shows how the IT systems meets the business goals of the enterprise

© 2014 Hans-Peter Hoidn & Kai Schwidder

Information Systems Architecture – Objectives (TOGAF 11.1)

- Develop the Target Application Architecture that enables the Business Architecture and the Architecture Vision, while addressing the Request for Architecture Work and stakeholder concerns

- Identify candidate Architecture Roadmap components based upon gaps between the Baseline and Target Application Architectures

© 2014 Hans-Peter Hoidn & Kai Schwidder

Information Systems Architecture – Steps (TOGAF 11.4)

- Select reference models, viewpoints, and tools
  - Determine Overall Modeling Process
  - Identify Required Catalogs of Application Building Blocks
  - Identify Required Matrices
  - Identify Required Diagrams
- Develop Baseline Application Architecture Description
- Develop Target Application Architecture Description
- Perform gap analysis (see Section 11.4.4)
- Define candidate roadmap components
- Resolve impacts across the Architecture Landscape
- Conduct formal stakeholder review
- Finalize the Application Architecture
- Create Architecture Definition Document

© 2014 Hans-Peter Hoidn & Kai Schwidder



© 2014 Hans-Peter Hoidn & Kai Schwidder

SOA (Service Oriented Architecture)

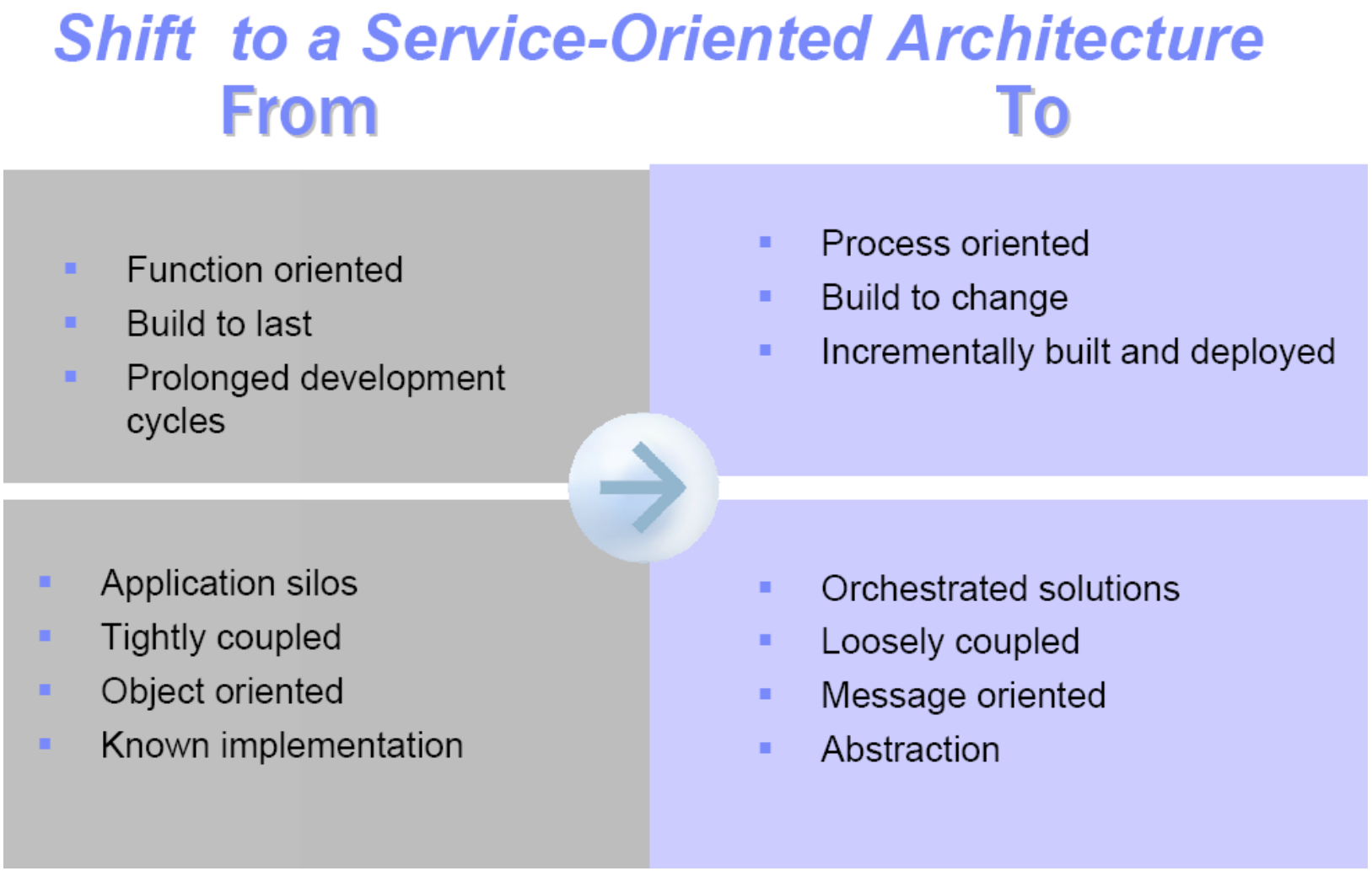
© 2014 Hans-Peter Hoidn & Kai Schwidder

What is SOA

- SOA is an *architectural style* or approach whose goal is to achieve loose coupling among interacting software agents
- All functions (that need to be used by more than one system) are defined as "*services*“
- Service providers agree to a defined, implementation-independent interface with service clients
- Services oriented architecture is the *policies, practices and frameworks*
  - that enable application functionality and IT services to be
  - provided and requested as a set of services
  - using a standards based form of interface.

© 2014 Hans-Peter Hoidn & Kai Schwidder

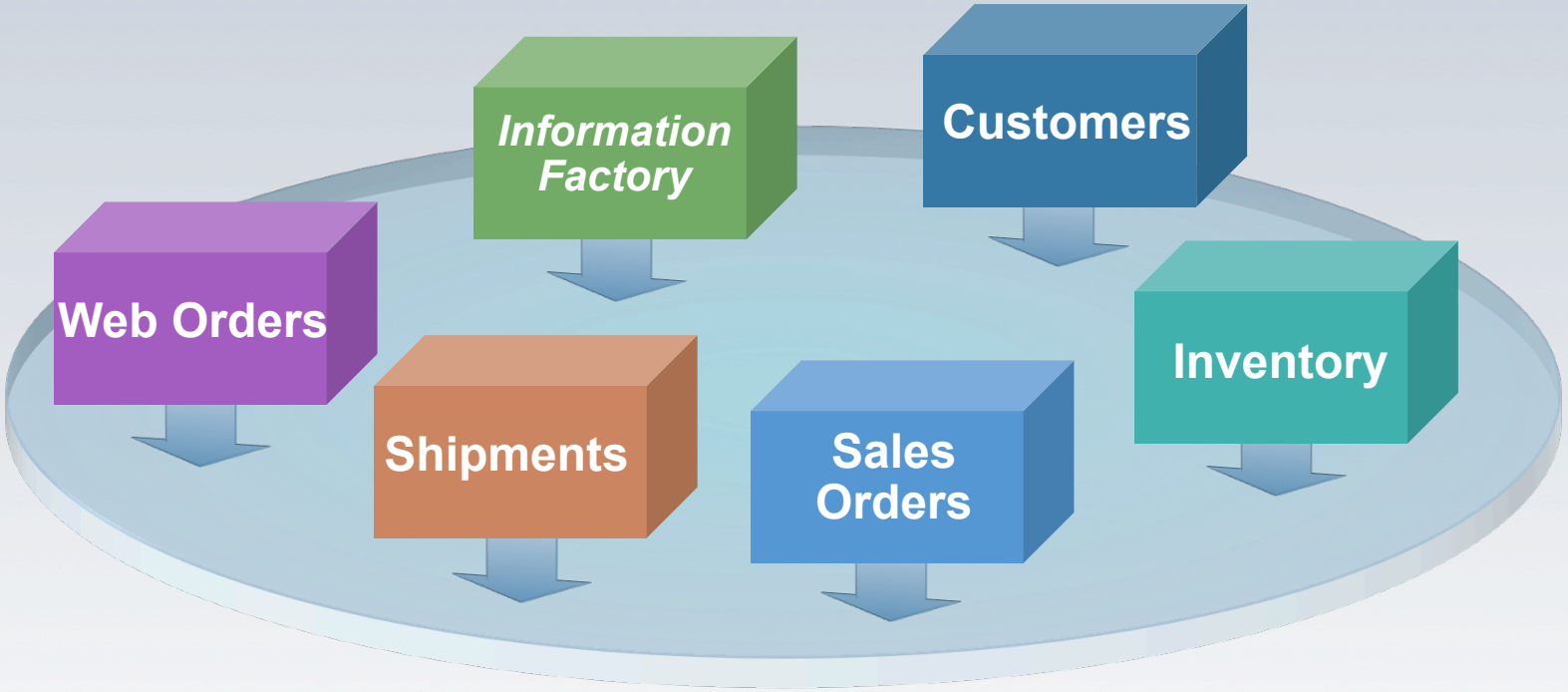
Observation of Common Shift



© 2014 Hans-Peter Hoidn & Kai Schwidder

Service Oriented Architecture (SOA)

Moves IT Logic Out of Services



Services defined as units of business logic separated from…

- Flow of control and routing
- Data transformation and protocol transformation

© 2014 Hans-Peter Hoidn & Kai Schwidder

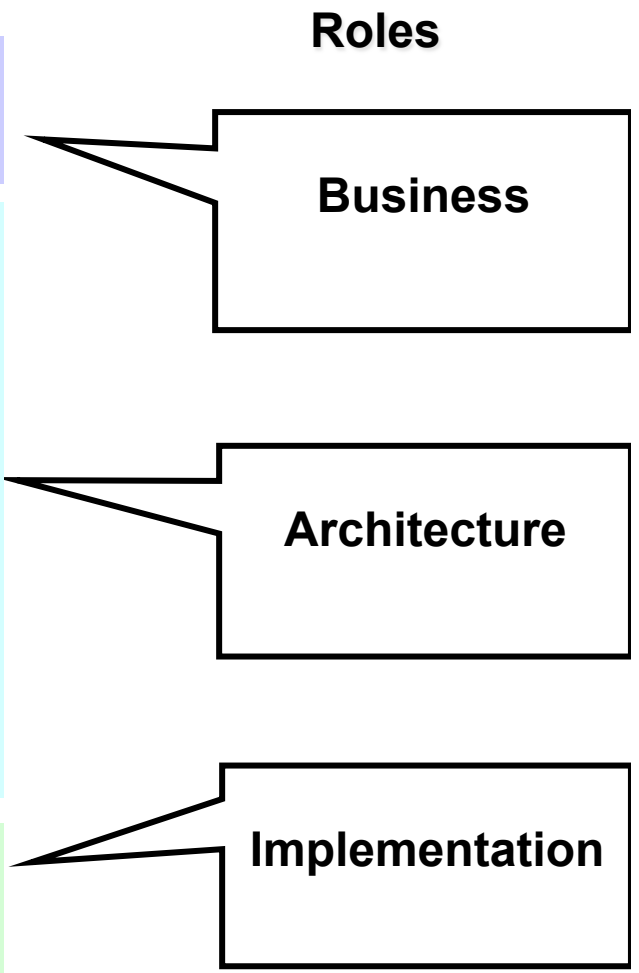
SOA is different things to different people

A set of services that a business wants to expose to customers and clients

an architectural style which requires a service provider, requestor and a service description.

a set of architectural principles and patterns which address characteristics such as *modularity, encapsulation, loose coupling, separation of concerns, reuse, composable and single implementation*.

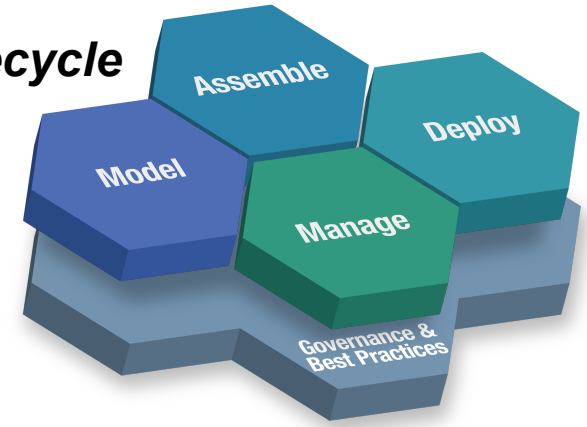
A programming model complete with standards, tools, methods and technologies such as web services.



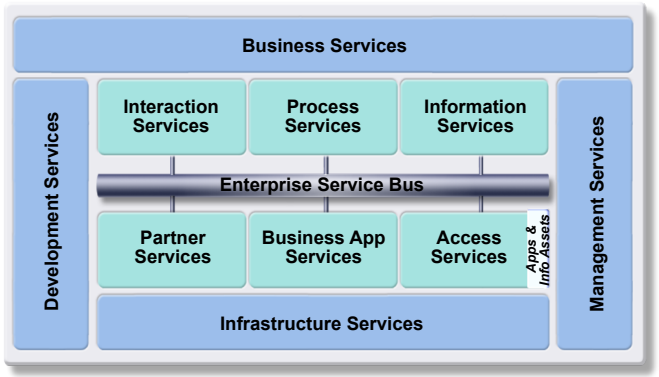
© 2014 Hans-Peter Hoidn & Kai Schwidder

Key Models and Methods for SOA – Enabling greater flexibility in Enterprise IT Architectures

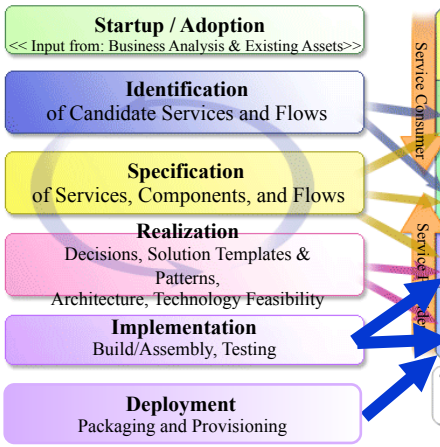
SOA Lifecycle



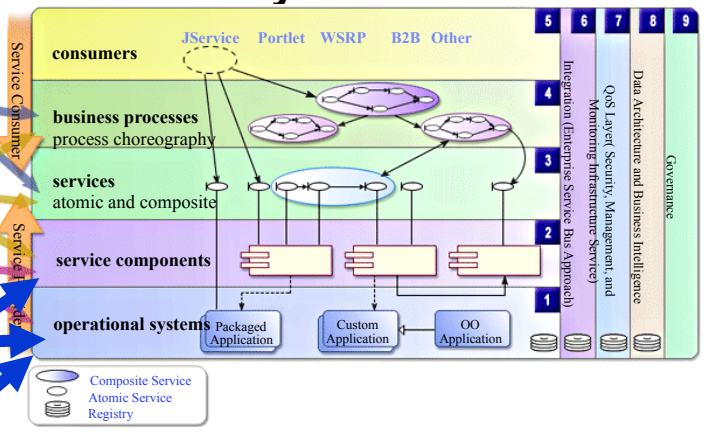
SOA Reference Architecture



The SOMA Method: Service-Oriented Modeling and Architecture



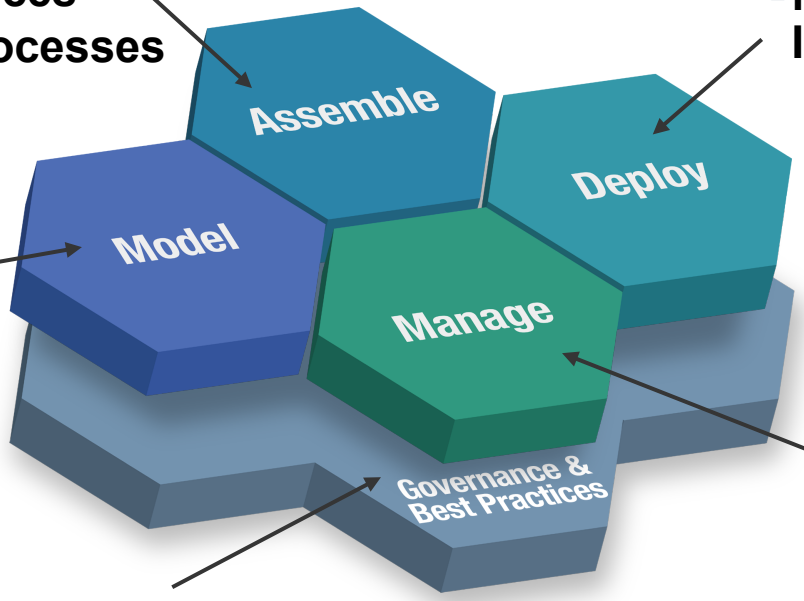
The SOA Solution Stack: Layered solution view



The SOA Lifecycle (to be addressed in detail in Governance)

- Discover Services
- Construct & Test Services / UIs
- Compose Services
- Orchestrate Processes

- Integrate People
- Integrate Processes
- Manage and integrate Information



- Gather & Model Requirements
- Model & Simulate
- Model SW Architecture

- Financial Transparency
- Business/IT Alignment
- Process Control

- Manage Applications & Services
- Manage Identity & Compliance
- Monitor Business Metrics

© 2014 Hans-Peter Hoidn & Kai Schwidder

Service Model

© 2014 Hans-Peter Hoidn & Kai Schwidder

Service Model – Overview

- The Service Model is in the center of the work products of an SOA based architecture
- There are several specifications about SOA and Services around – will need some more investigation
- Note:
  - Service Model is a result of SOMA (see SOA part)
  - Portfolio of services is replacing the application landscape
- Relevant for Short Term Engagement:
  - Identification of services
  - Realization options

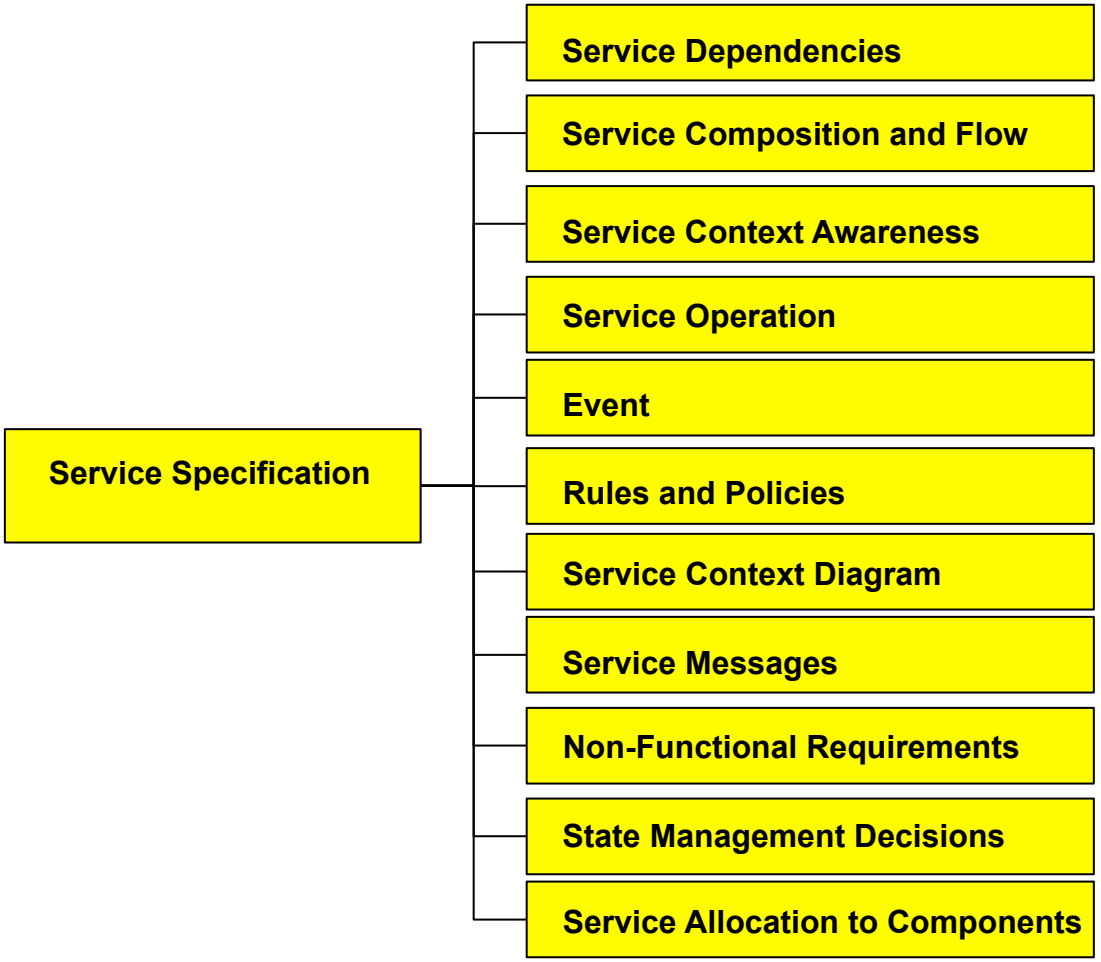
© 2014 Hans-Peter Hoidn & Kai Schwidder

OMG's SoaML Specification (according to ad-08-09-12) Goals

- Intuitive and complete support for modeling services in UML
- Support for bi-directional asynchronous services between multiple parties
- Support for Services Architectures where parties provide and use multiple services.
- Support for services defined to contain other services
- Easily mapped to and made part of a business process specification
- Compatibility with UML, BPDM and BPMN for business processes
- Direct mapping to web services
- Top-down, bottom up or meet-in-the-middle modeling
- Design by contract or dynamic adaptation of services
- To specify and relate the service capability and its contract
- No changes to UML

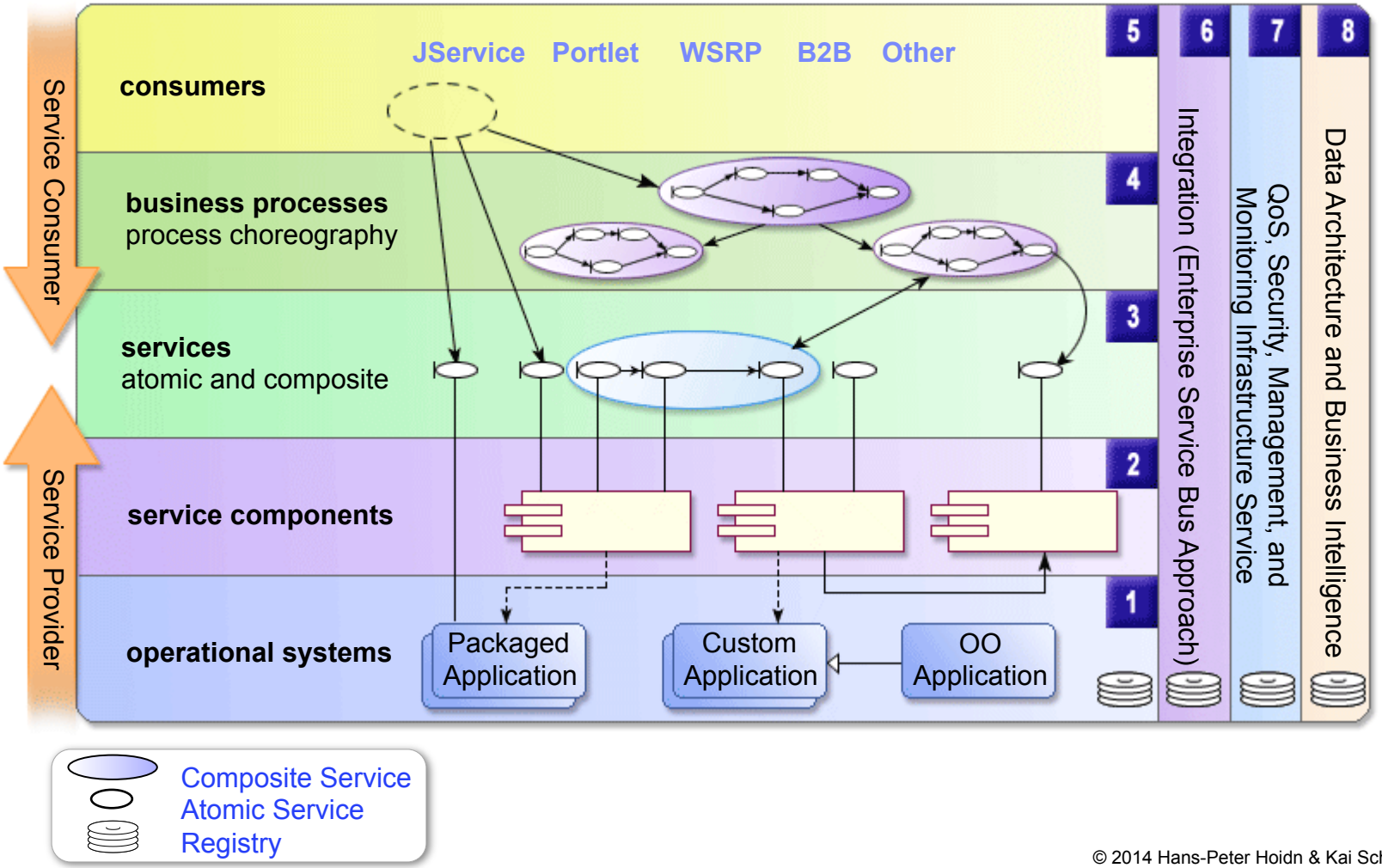
© 2014 Hans-Peter Hoidn & Kai Schwidder

Service Specification – Services are exposed by Components



© 2014 Hans-Peter Hoidn & Kai Schwidder

SOA Layered View (Solution Stack)



© 2014 Hans-Peter Hoidn & Kai Schwidder

SOA Technology

© 2014 Hans-Peter Hoidn & Kai Schwidder

Definition “Service”

- **Business Perspective: A Service**
  - **is a well- defined, encapsulated, reusable, business-aligned capability.**
  - **is fully defined by a service description, a published document or artifact that outlines the overall objective of the service and its inputs, purpose, outputs, scope, responsibility, governance, sustainability (provision period, maintenance, and repair), and qualities of service provisioning**

- **IT Perspective: A Service**
  - **is a discoverable, invokable software resource that has a service description and interface and is configurable using policies.**
  - **implementation is realized through a service provider that delivers quality of service (QoS) requirements for the service consumer**

© 2014 Hans-Peter Hoidn & Kai Schwidder

SOAP

- SOAP is a protocol specification for exchanging structured information in the implementation of web services

- Its XML-based protocol consists of three parts:
  - an envelope, which defines the message structure
  - a set of encoding rules
  - a convention for representing procedure calls and responses

- SOAP has three major characteristics:
  - extensibility (security and WS-routing are among the extensions under development)
  - neutrality (SOAP can operate over any transport protocol such as HTTP, SMTP, TCP, UDP, or JMS)
  - independence (SOAP allows for any programming model)

© 2014 Hans-Peter Hoidn & Kai Schwidder

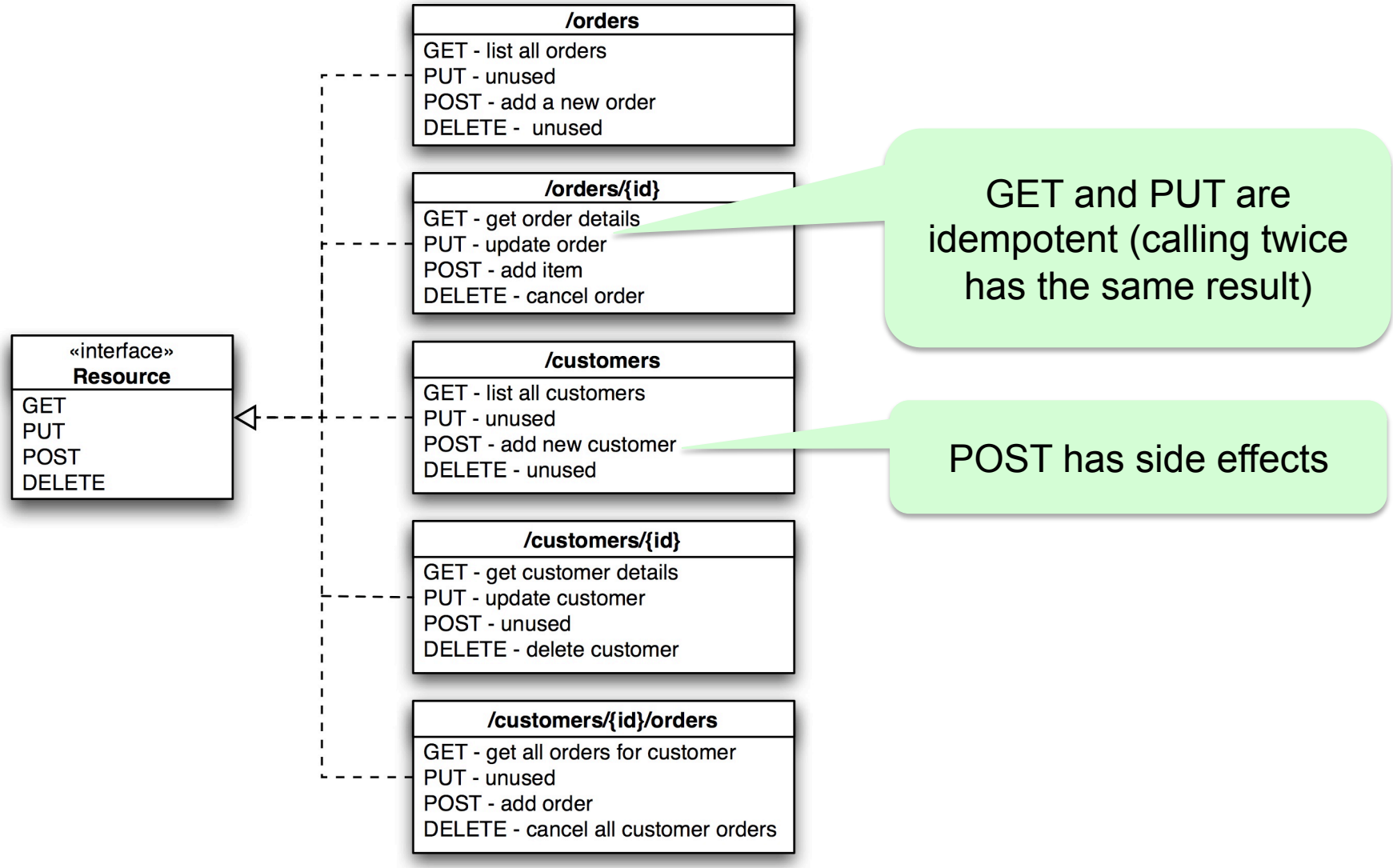
REST (Representational State Transfer)

- REST is providing a generic interface, such that with REST an application is treated as a web application
- HTTP is the transport protocol. URIs access resources, a resource becomes a web resource by simply making the information associated with it accessible to the web
- The basic functions on the resources (entities as well as other "things" like a process) are GET, POST, PUT and DELETE
  - GET, PUT, DELETE are idempotent (no harm when called twice)
  - POST has side effects (may also on other resources)
- The advantage is that the lean data format JSON can be used

Source: Roy Fielding defined REST in the year 2000 in his thesis

© 2014 Hans-Peter Hoidn & Kai Schwidder

Example RESTful Services



© 2014 Hans-Peter Hoidn & Kai Schwidder

ESB (Enterprise Service Bus) – Definition and Purpose

- An Enterprise Service Bus (ESB) is an architectural pattern defining a flexible connectivity infrastructure for integrating applications and services.

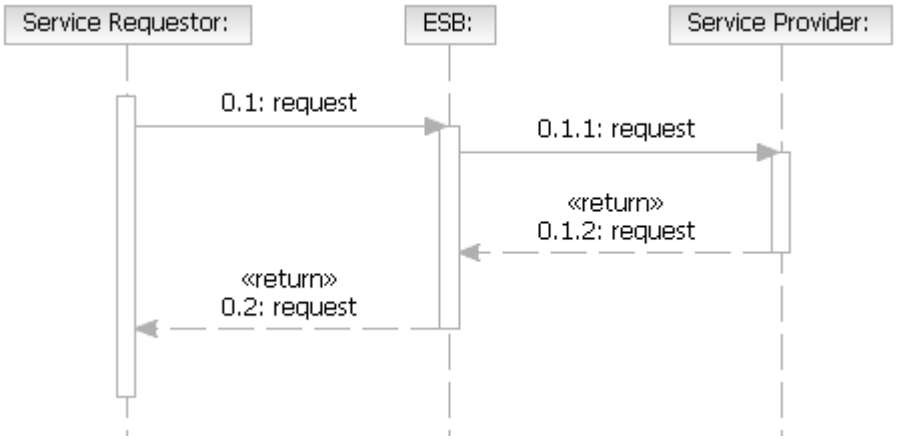
- The architecture pattern is a guiding principle to enable the integration and federation of multiple service bus instantiations.

- An ESB performs:
  - Routing messages between services
  - Converting transport protocols between requestor and service – managing multiple protocols
  - Transforming message content between requestor and service
  - Handling business events from disparate sources

© 2014 Hans-Peter Hoidn & Kai Schwidder

ESB (Enterprise Service Bus) – Service Virtualization
Decoupling of Service Requester and Provider

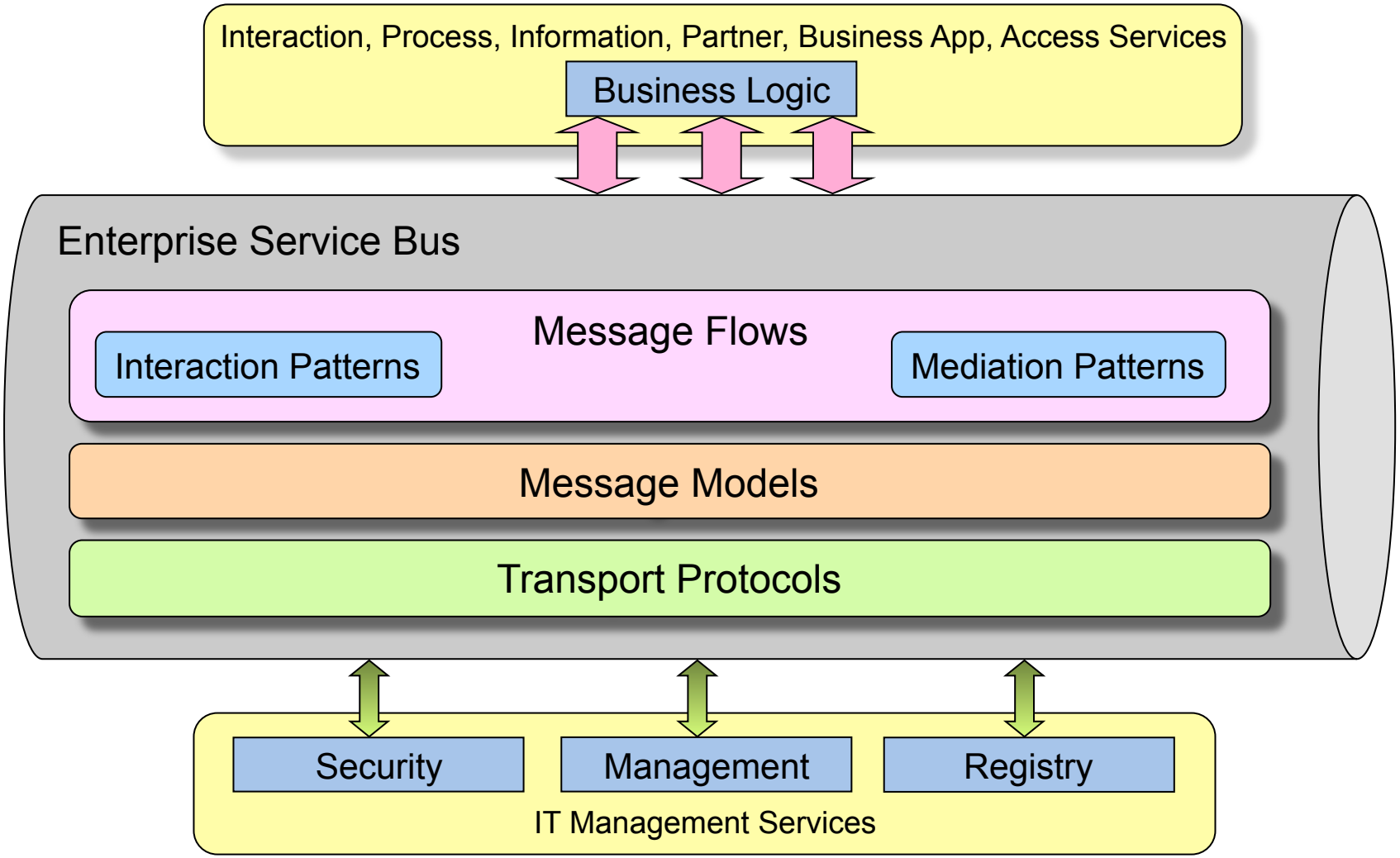
- **ESB acts as an intermediary (proxy) between requestor and provider**



- **ESB provides *service virtualization* of**
  - ***Location and identity***
  - ***Interaction protocol***
  - ***Interface***
- **Interactions are *decoupled*, supporting *separation of concerns***

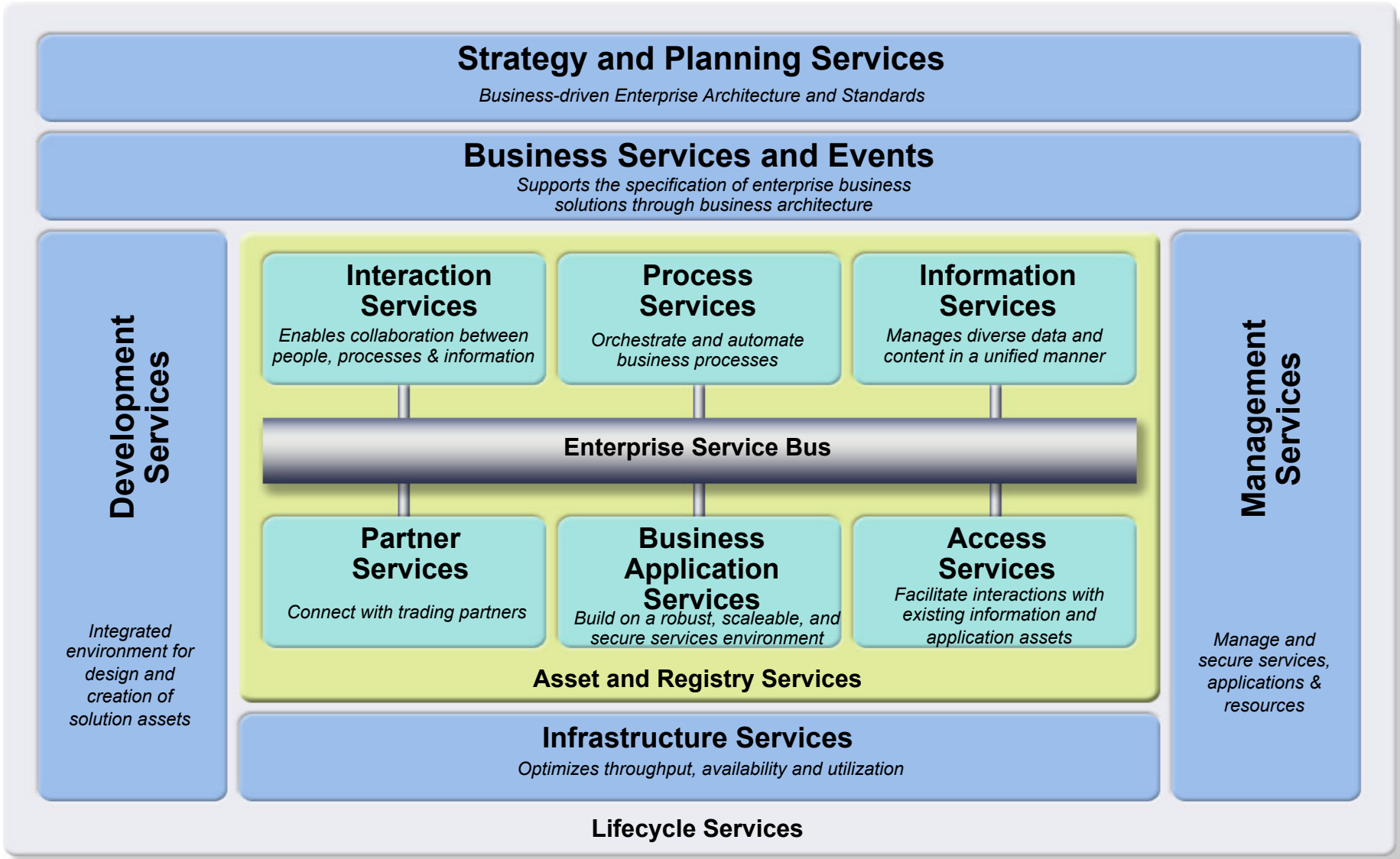
© 2014 Hans-Peter Hoidn & Kai Schwidder

Expanded View of the Enterprise Service Bus



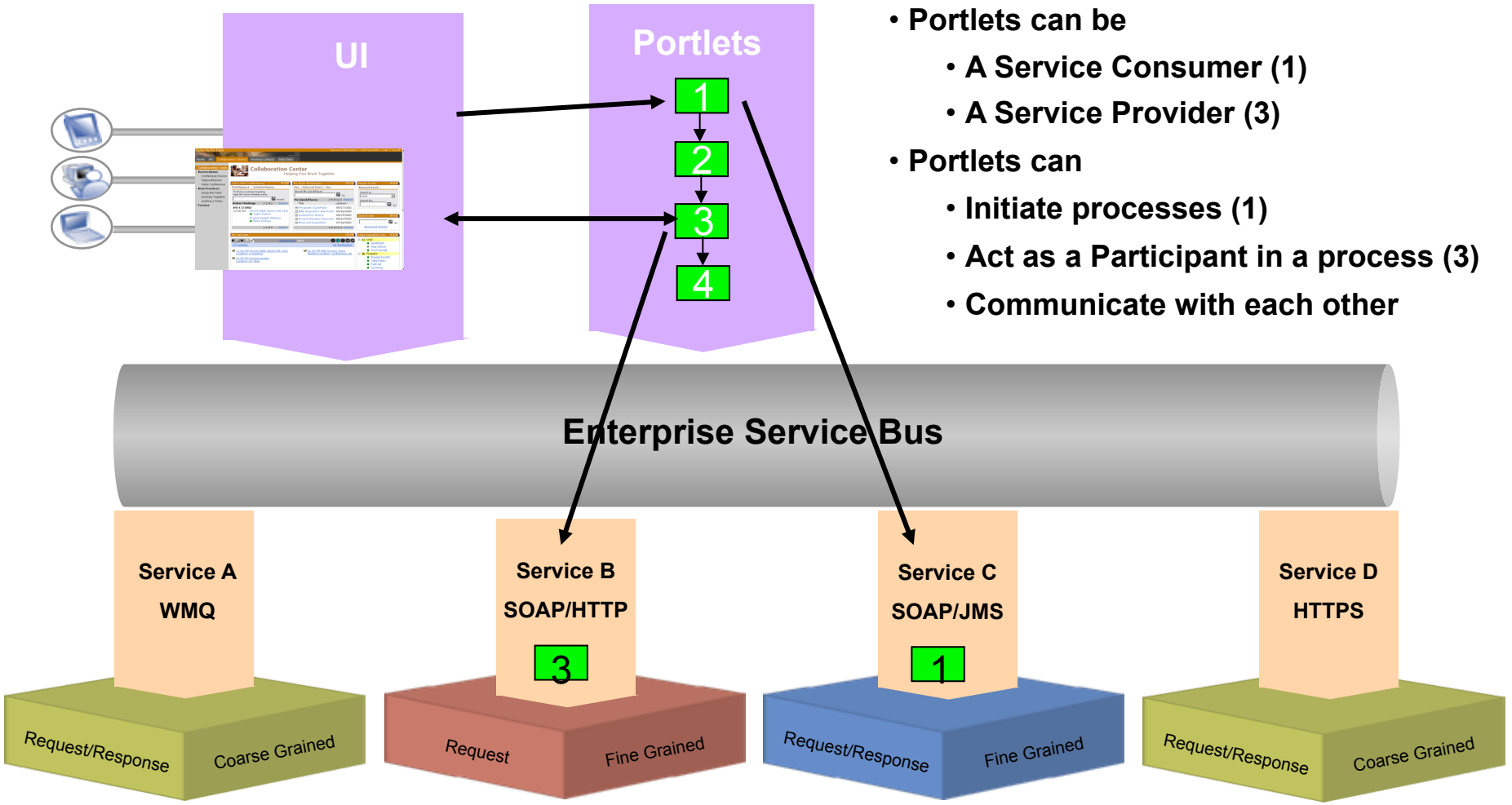
© 2014 Hans-Peter Hoidn & Kai Schwidder

IBM SOA Foundation Reference Model (Key Concepts)



© 2014 Hans-Peter Hoidn & Kai Schwidder

What is an *Interaction Service*?

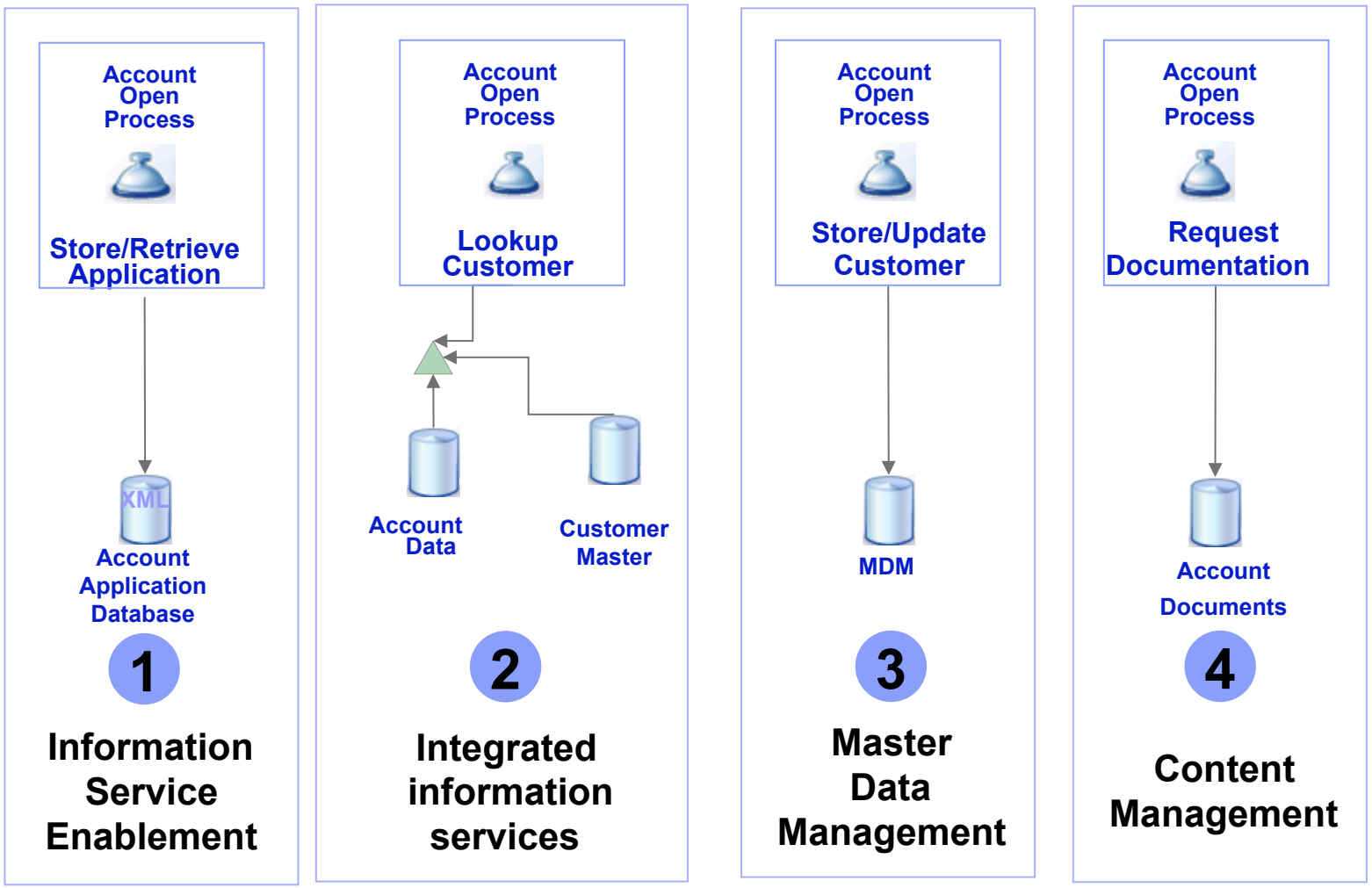


- Portlets can be
  - A Service Consumer (1)
  - A Service Provider (3)
- Portlets can
  - Initiate processes (1)
  - Act as a Participant in a process (3)
  - Communicate with each other

The Portal Framework Provides Service Aggregation

© 2014 Hans-Peter Hoidn & Kai Schwidder

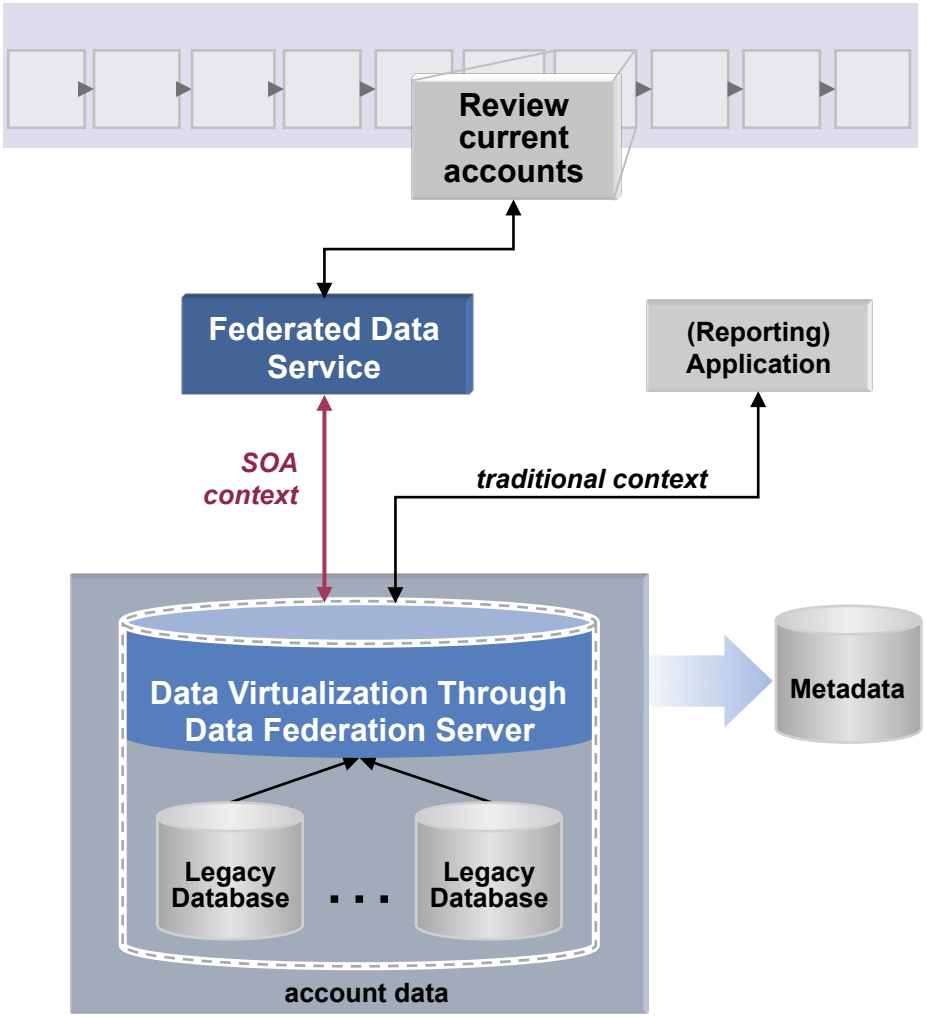
Information Services: Several Patterns



© 2014 Hans-Peter Hoidn & Kai Schwidder

Information Services: Pattern – Deliver Your Data *Virtualized Through Services*

- **As-Is Environment**
  - Data resides in disparate sources
  - Manual & redundant integration of data by multiple consumers results in high costs and inconsistent/ inaccurate data
  - Slow response time due to inefficient real-time access
- **Solution Characteristics**
  - On demand integration instead of redundant data
  - Transparent & optimized access to distributed, heterogeneous sources
- **Results**
  - Real-time access to distributed information, fast response time
  - Scalable approach for adding more data sources

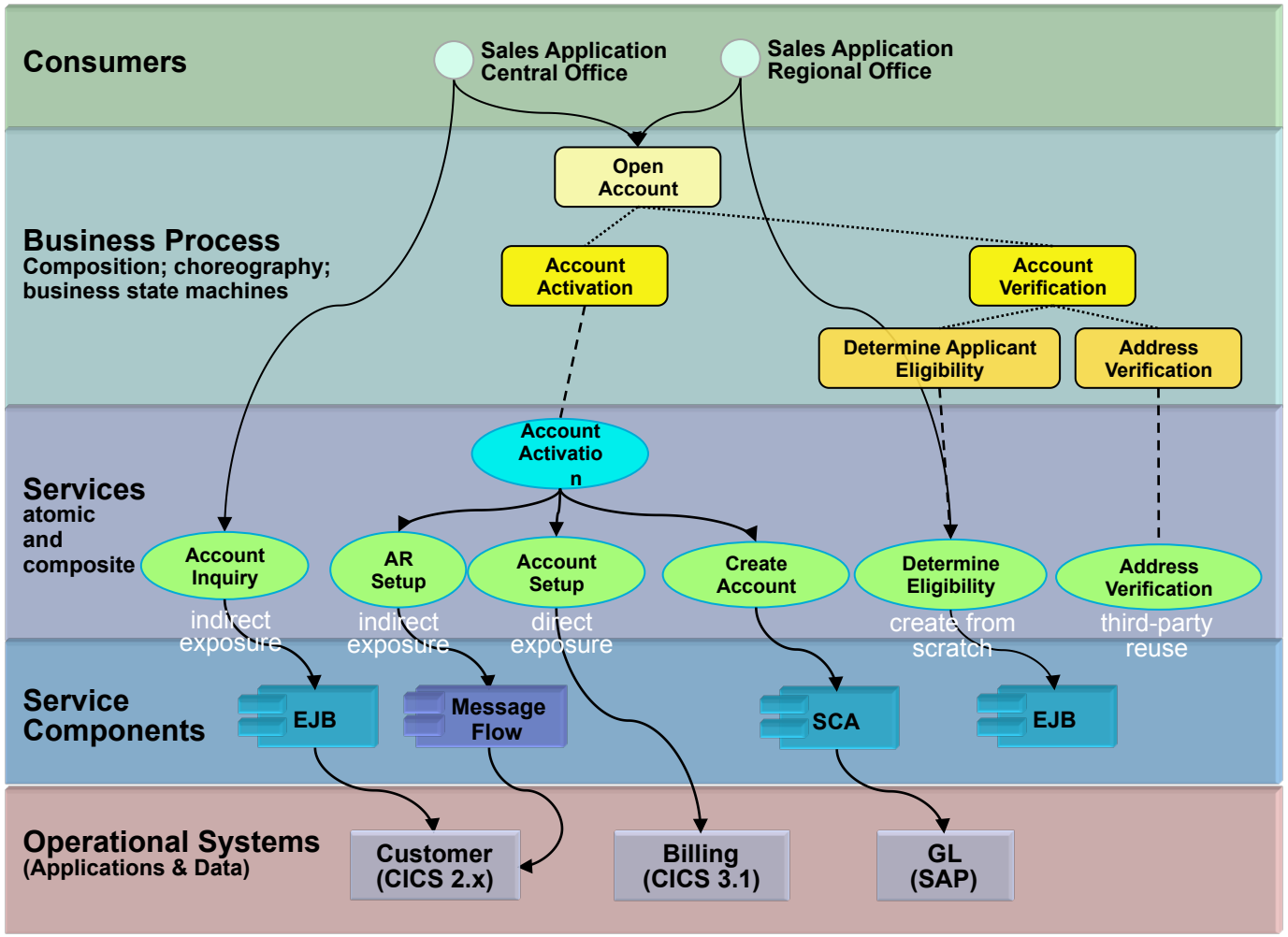


© 2014 Hans-Peter Hoidn & Kai Schwidder

SOA Layered View Details

© 2014 Hans-Peter Hoidn & Kai Schwidder

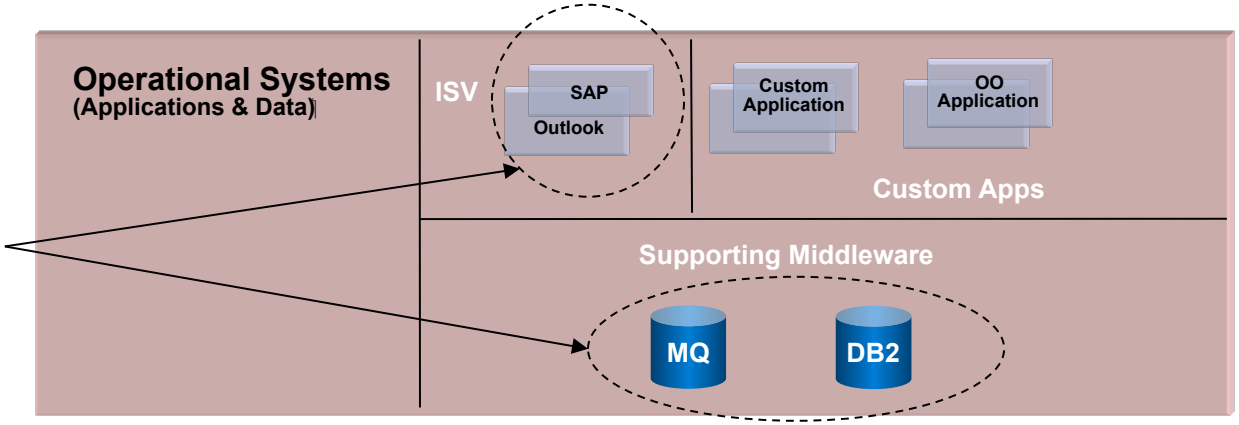
Example JK Enterprise – a virtual company with an „Open Account Process“



© 2014 Hans-Peter Hoidn & Kai Schwidder

Layer 1: Operational Systems (Leverage Existing Investment)

Examples for illustration: specifics are not in the scope of the reference model.

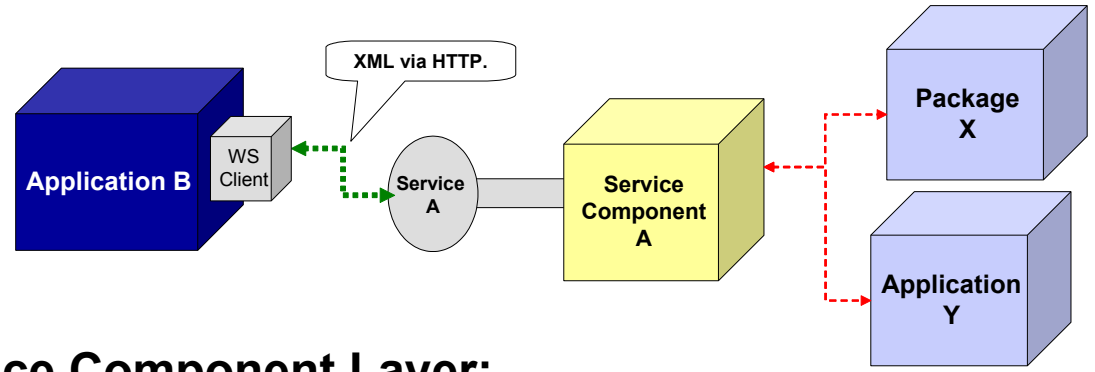


- **Recognizes the value of existing IT investment**
  - Use of existing "legacy" applications (e.g. COBOL application) and / or packages (e.g. SAP)
- **Some SOA Related Activities:**
  - Asset Inventory
  - Refactor existing applications to unlock business value

© 2014 Hans-Peter Hoidn & Kai Schwidder

Layer 2: Service Components

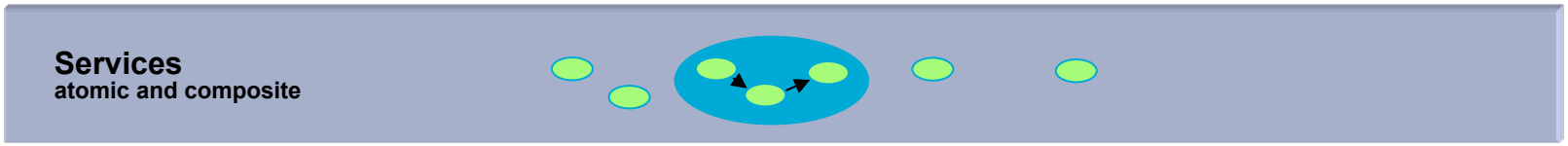




- **The Service Component Layer:**
  - **Enables IT flexibility by strengthening the decoupling in the system. Decoupling is achieved by hiding volatile implementation details from consumers.**
  - **Often employs container based technologies like EJBs**
- **Each Service Component:**
  - **Provides an enforcement point for service realization**
  - **Offers a facade behind which IT is free to do what they want/need to do**

© 2014 Hans-Peter Hoidn & Kai Schwidder

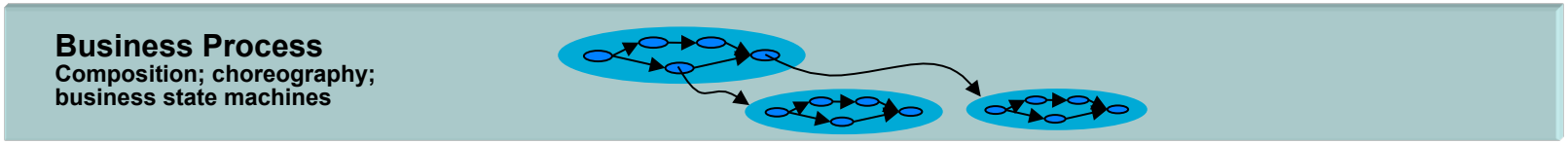
Layer 3: Services (Decouple Business and IT)



- **The Services Layer forms the basis for the decoupling of Business and IT.**
  - **Captures the functional contract (incl. QoS – Quality of Service) for each standalone *business* function or each task in a business process**
- **The assumption is that (within an SOA) IT responsibility is to realize/ manage service implementations that faithfully conform to the set of services in the service model.**
- **This layer contains all the *exposed services* in the SOA**
- **Each service is a contract between the consumer(s) and the provider(s)**

© 2014 Hans-Peter Hoidn & Kai Schwidder

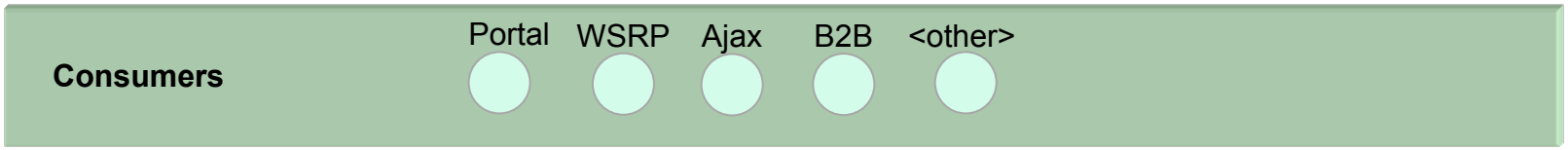
Layer 4: Business Processes (Business process alignment of IT)



- This layer contains operational IT artifacts that implement business processes as a choreography of services
- The set of services that are composed is restricted to those services that are defined in Layer 3
- The choice of technology depends on a set of realization decisions that must be made when establishing a physical Reference Model for a given SOA

© 2014 Hans-Peter Hoidn & Kai Schwidder

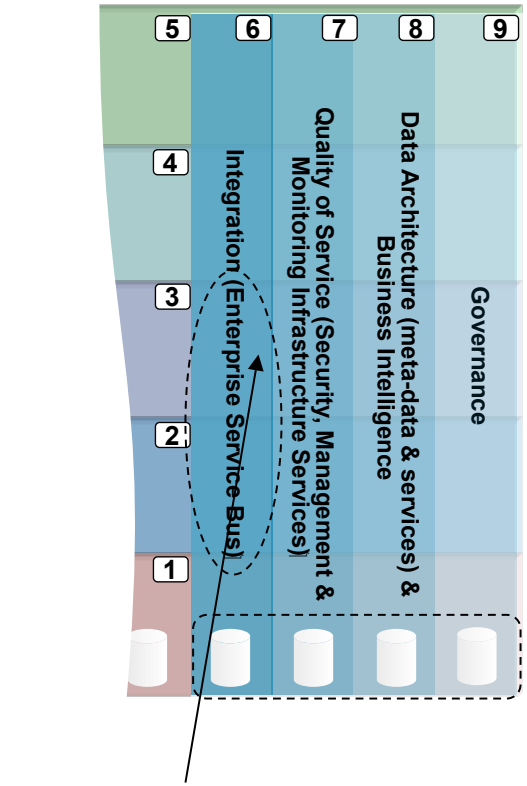
Layer 5: The Consumer Layer
(Channel independent access to business processes)



- **This layer exists to recognize that the technology chosen to expose Business Processes/Services must permit access from a wide set of interaction *channels*.**
- **It is important to populate this layer with the set of *channels* types that are required in a solution.**

© 2014 Hans-Peter Hoidn & Kai Schwidder

Cross-cutting concerns/capabilities



- Several concerns are not restricted to a single layer in the Reference Model, these concerns are captured in ‘Layers’ 6-9
- These are not really layers but treating them as such gives us the ability focus discussions/ decisions, for example “What is found where Governance intersects Services? i.e. what are the Governance concerns specific to Services?”
- Clearly there is interaction among these ‘layers’ also. For example, it is likely that most data architectures will be subject to governance

© 2014 Hans-Peter Hoidn & Kai Schwidder



© 2014 Hans-Peter Hoidn & Kai Schwidder